

The image shows a well-lit industrial networking lab. In the foreground, there is a long wooden workbench on a black frame with many drawers. A black ergonomic chair is positioned in front of it. In the background, several rows of black equipment racks are filled with various electronic components and cables. A projector is mounted on the ceiling, and the room has a drop ceiling with fluorescent lights.

INDUSTRIAL NETWORKING

MENG 4010

CONTENTS

9B. OPC UA

- 9B.1. Introduction to Industrial Communication Systems
- 9B.2 Introduction to OPC UA
- 9B.3 OPC UA - System Architecture
- 9B.4 OPC UA - Address Space and Information Models
- 9B.5 OPC UA - Security Model
- 9B.6 OPC UA - Services
- 9B.7 OPC UA – Information Model

9B.1 Introduction to Industrial Communication Systems

- An industrial communication system is a network that allows industrial devices such as PLCs, sensors, actuators, HMIs, drives, and SCADA systems to exchange data efficiently and reliably.
- These networks differ from office or enterprise networks (like standard Ethernet used for web browsing or email) because they must handle:
 - Real-time data exchange
 - High reliability and determinism
 - Noise immunity
 - Long-distance communication
 - Compatibility with legacy systems

Layer	Description	Examples
Enterprise	Business management, planning, and analytics	ERP, MES
Supervisory	Process monitoring and control	SCADA, HMI
Control	Direct machine and process control	PLCs, PACs
Field	Data acquisition and actuation	Sensors, actuators, drives



9B.1 Introduction to Industrial Communication Systems



- Early Automation (Proprietary Point-to-Point Systems): Devices used manufacturer-specific protocols with unique cabling and configurations, leading to poor interoperability, difficult multi-vendor integration, and high maintenance/scalability costs.
- Rise of Fieldbus Networks (1980s-1990s): Digital, multidrop networks like PROFIBUS, Modbus RTU, DeviceNet, CANopen, and Foundation Fieldbus reduced wiring, enabled bidirectional communication, and improved diagnostics/integration, but faced proprietary variations, limited bandwidth, and IT integration challenges.
- Transition to Industrial Ethernet: Adopted for high-speed Ethernet with real-time protocols (e.g., EtherNet/IP, PROFINET, EtherCAT, Modbus TCP) to ensure determinism; offers high bandwidth, seamless enterprise IT integration, TCP/IP support, and enhanced scalability/configuration.

Characteristic	Description
Real-time communication	Data must be transmitted within strict time limits.
Determinism	Predictable timing of data delivery.
Reliability	Continuous operation with minimal downtime.
Scalability	Ability to connect multiple devices and networks.
Interoperability	Devices from different vendors must work together.
Security	Protection against unauthorized access or data manipulation.

9B.1 Introduction to Industrial Communication Systems



Industrial communication evolved rapidly, but a major challenge persisted: interoperability.

Devices from different vendors often used incompatible communication protocols.

Organizations and standards bodies such as IEC (International Electrotechnical Commission) and OPC Foundation worked to define open communication standards.

Key standardization benefits:

- Interoperability between devices from multiple vendors
- Reduced integration and maintenance costs
- Easier system scalability
- Long-term technology sustainability

The OPC (OLE for Process Control) standard emerged as one of the key solutions to ensure vendor-neutral data exchange in industrial environments.

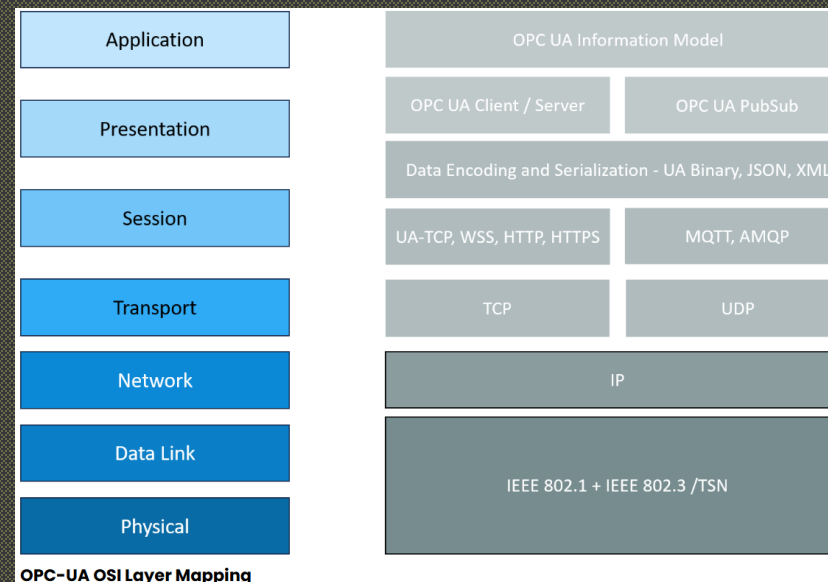
This led to the development of OPC UA, which extends beyond the limitations of the earlier OPC Classic model - offering cross-platform compatibility, improved security, and rich data modeling capabilities.

9B.1 Introduction to Industrial Communication Systems



- OPC (Open Platform Communications) is a platform-independent standard for secure and reliable data exchange in industrial automation. It allows devices from different vendors to communicate seamlessly.
- Originally known as OLE (Object Linking and Embedding) for Process Control, OPC started on Windows to unify communication with PLCs and HMIs.
- The newer OPC UA (Unified Architecture) supports modern needs like cross-platform compatibility, security, and scalability, making it suitable for future industrial applications. The OPC Foundation develops and maintains these standards.

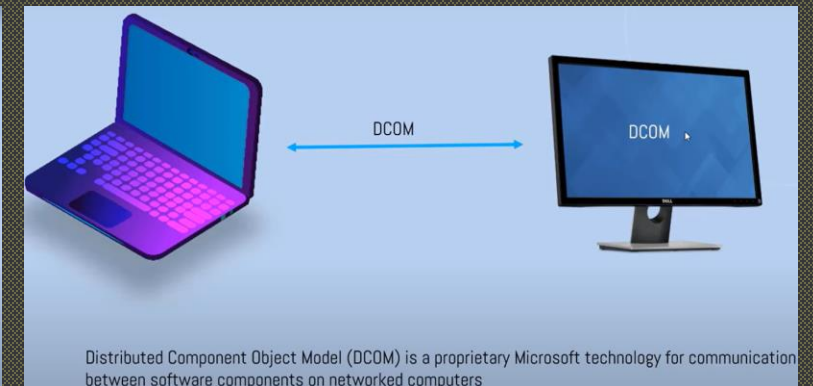
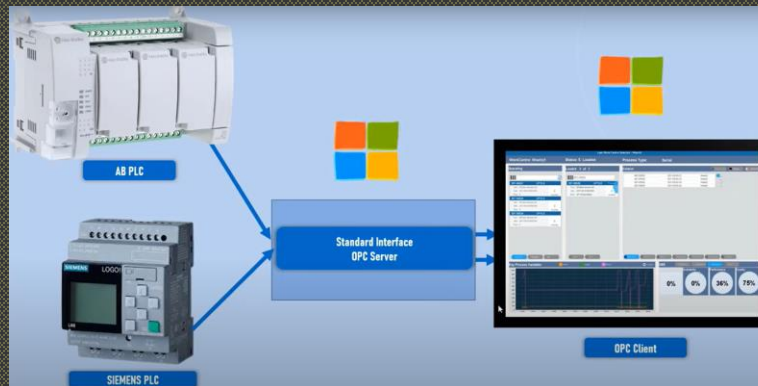
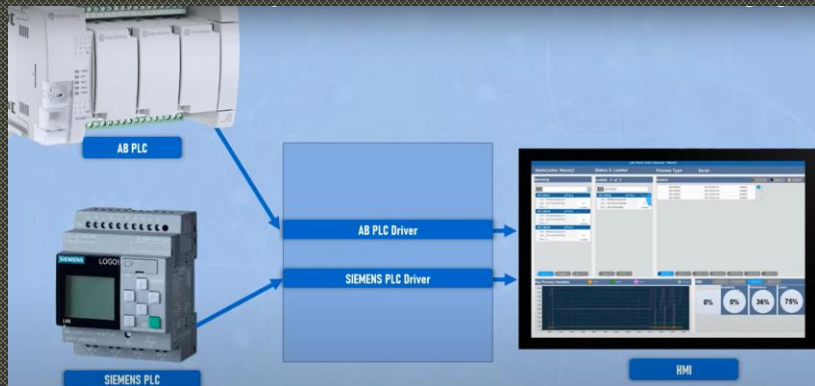
A&E - Alarms and Events
COM - Component Object Model
DCOM – Distributed Component Object Model
DA - Data Access
DCS - Distributed Control System
DDS - Data Distribution Service
HDA - Historical Data Access
MES - Manufacturing Execution System
ERP – Enterprise Resource Planning
OPC - Open Platform Communications
UA - Unified Architecture
WSS – WebSocket Secure



AMQP - Advanced Message Queuing Protocol
API - Application Programming Interface
JSON - JavaScript Object Notation
LDAP - Lightweight Directory Access Protocol
MQTT - Message Queue Telemetry Transport
OAuth2 - Open Authorization
UADP - UA Datagram Protocol
WSDL - Web Services Definition Language
XML - Extensible Markup Language
XMPP - Extensible Messaging and Presence Protocol

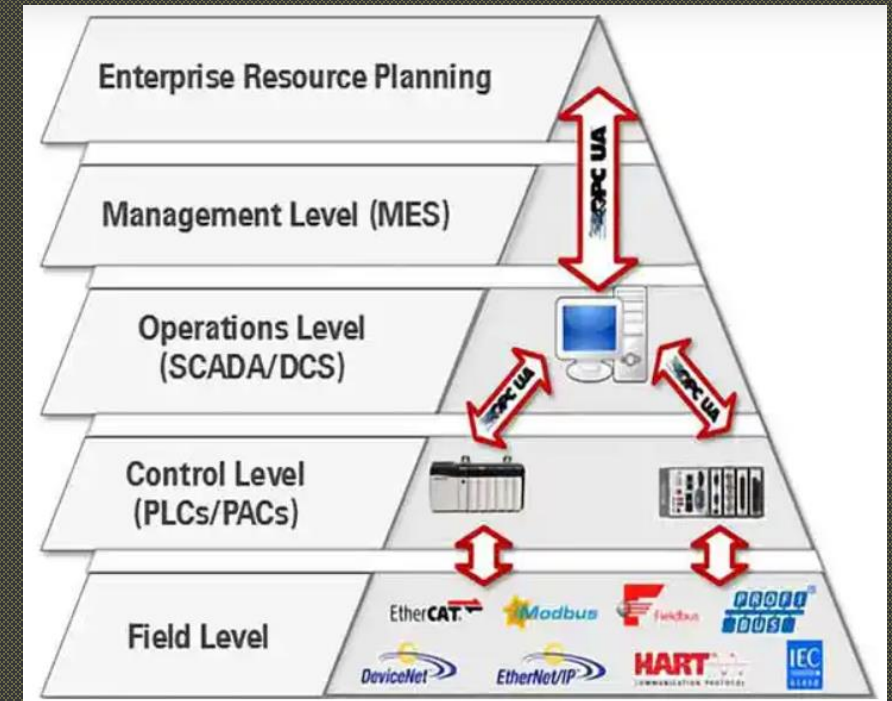
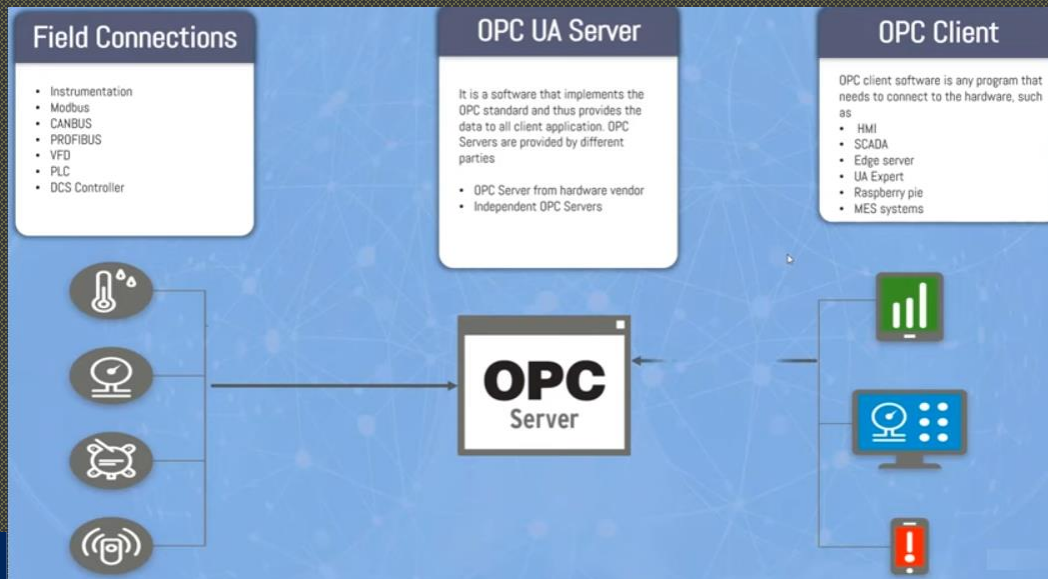
9B.2 Introduction to OPC UA

- OPC Classic, introduced in 1996, is a widely used standard for reliable data exchange in industrial automation, using Microsoft's COM/DCOM technologies.
- It includes specifications like OPC DA (real-time data), HDA (historical data), and A&E (alarms/events). However, it requires separate OPC servers per device, which can complicate systems.
 - Each device (like a PLC or sensor) typically needs its own OPC Server.
 - Managing several servers becomes complex, harder to maintain, and limits system scalability.
- OPC UA, the next-generation version, overcomes these limits with better scalability, cross-platform support, cloud readiness, and improved security - enabling seamless, future-proof interoperability in modern IIoT environments.



9B.2 Introduction to OPC UA

- OPC UA is a universal standard for enabling different industrial devices and systems to communicate and work together - regardless of the manufacturer or platform.
- It applies across all industrial levels:
 - Sensors and actuators (on the shop floor)
 - Control systems (like PLCs)
 - MES (Manufacturing Execution Systems)
 - ERP (Enterprise Resource Planning)
 - IIoT and M2M systems



<https://tipteh.com/opc-ua/>

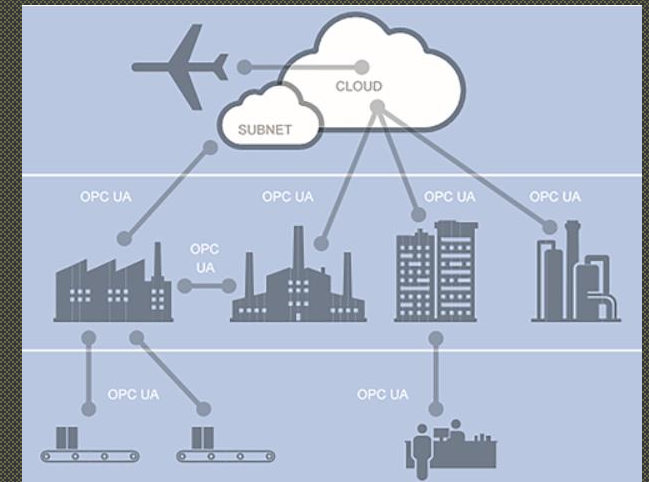
9B.2 Introduction to OPC UA

1. Architecture Overview

- OPC UA is a client-server architecture where clients (e.g., HMIs, apps) request data or actions from servers (e.g., PLCs, sensors). It is designed to work on any platform - Windows, Linux, embedded systems - using standard internet protocols like TCP/IP.
- Example: Imagine a factory where an HMI (client) asks a PLC (server) for a motor's speed. OPC UA ensures this works whether the PLC runs on a Raspberry Pi or a high-end PC.

2. Interoperability

- The goal is to make devices from different manufacturers (e.g., Siemens, Rockwell) work together. OPC UA uses a common data model and services, so a sensor from Company A can talk to software from Company B.
- Example: A temperature sensor using EtherCAT can share data with a cloud dashboard via OPC UA, even if they are made by different vendors.



<https://iebmedia.com/technology/opc-ua/best-practices-migrating-from-opc-classic-to-opc-ua/>

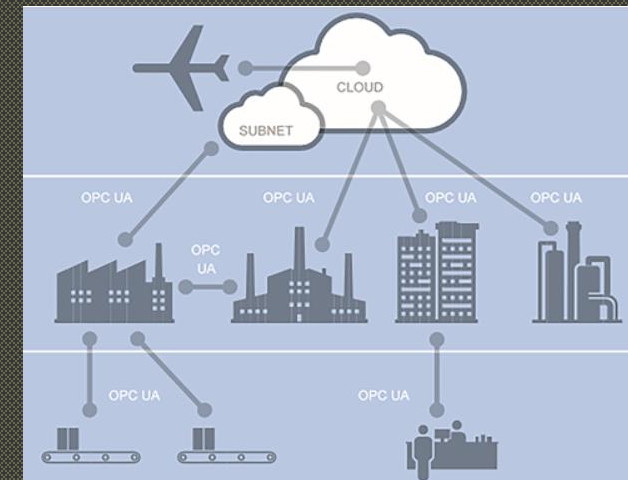
9B.2 Introduction to OPC UA

3. Security

- OPC UA has built-in security features like encryption, authentication, and authorization to protect data and prevent unauthorized access.
- Example: A remote operator can adjust a valve's settings safely because OPC UA encrypts the command, ensuring only the right person can do it.

4. Scalability

- The specification supports systems of all sizes, from a single machine to a global network, by allowing flexible data modeling and communication.
- Example: A small workshop might use OPC UA to monitor one robot, while a car plant uses it to connect thousands of devices across multiple sites.



9B.2 Introduction to OPC UA

5. Information Modeling

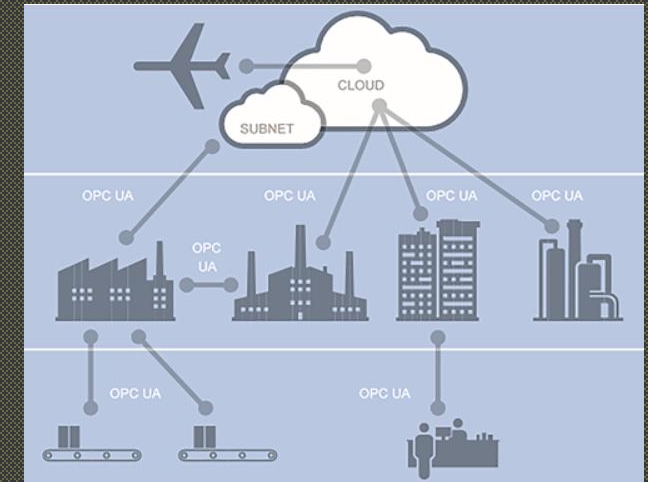
- OPC UA introduces the concept of an Address Space, a virtual map of all data and objects in the server (e.g., variables like temperature, methods like start/stop). This model can be customized for specific industries.
- Example: In a food processing plant, the Address Space might include nodes for "OvenTemperature" and "StartBaking," tailored to baking processes.

6. Service-Oriented Architecture (SOA)

- OPC UA uses services (e.g., Read, Write, Subscribe) to handle data exchange, making it flexible and future-proof.
- Example: A client subscribes to a pump's pressure updates, getting real-time alerts without constantly asking thanks to the Subscribe service.

7. Transport Independence

- OPC UA can run over different communication protocols (e.g., TCP, HTTP, WebSockets), adapting to the network environment.
- Example: A factory might use TCP for local PLCs and HTTP for cloud data, all handled by the same OPC UA system.



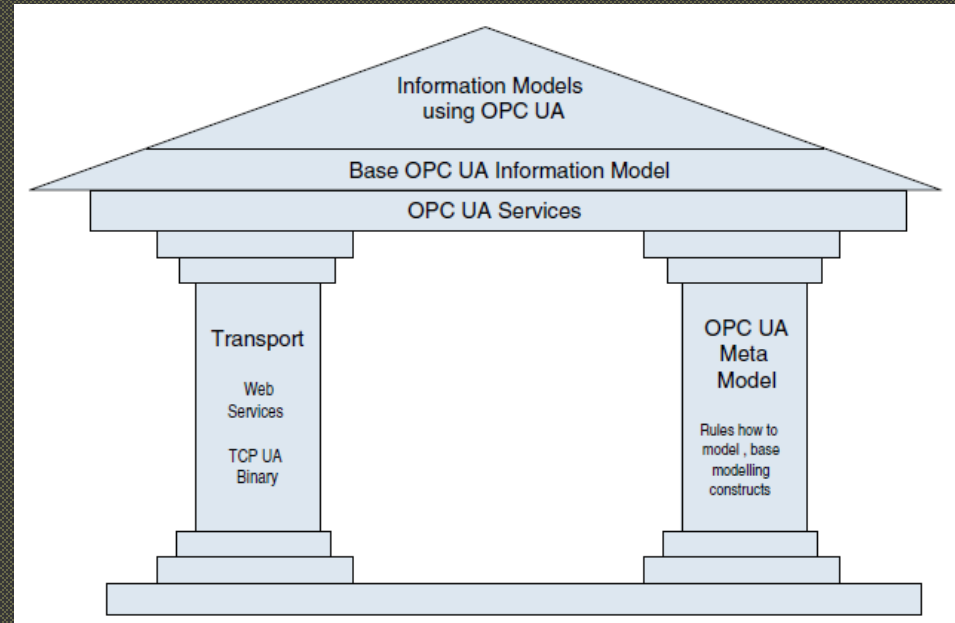
<https://iebmedia.com/technology/opc-ua/best-practices-migrating-from-opc-classic-to-opc-ua/>

The WebSocket protocol initiates a persistent, full-duplex communication channel over TCP via an initial HTTP handshake upgrade, facilitating low-latency, bidirectional data exchange ideal for real-time applications such as chat systems and live data streams.

9B.3 OPC UA - System Architecture

OPC UA is built on two core components: transport mechanisms and data modeling.

1. Transport Mechanisms: OPC UA supports different communication methods (transports) based on the needs of the application:
 - For high-performance intranet use (e.g., inside a factory), OPC UA uses a binary TCP protocol, which is fast and efficient.
 - For internet-based communication (e.g., remote access), it supports standard web technologies like Web Services, XML, and HTTP, which are more compatible with firewalls.
 - Despite using different transports, both methods share the same secure, message-based security model, ensuring consistent protection of data regardless of the communication method.

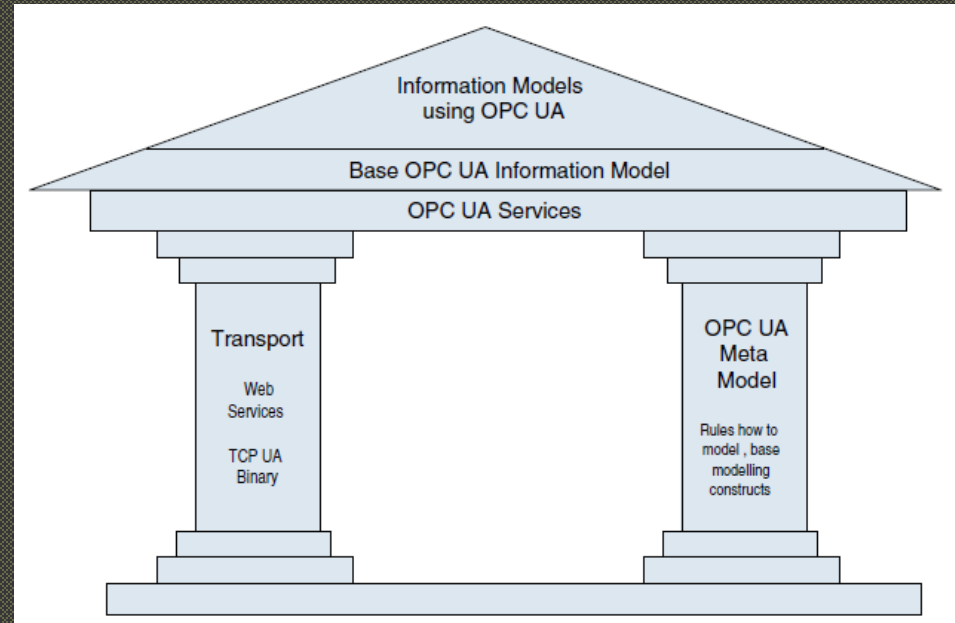


9B.3 OPC UA - System Architecture

OPC UA is built on two core components: transport mechanisms and data modeling.

2. Data Modeling: OPC UA handles data modeling, which is the foundation for organizing and sharing data.

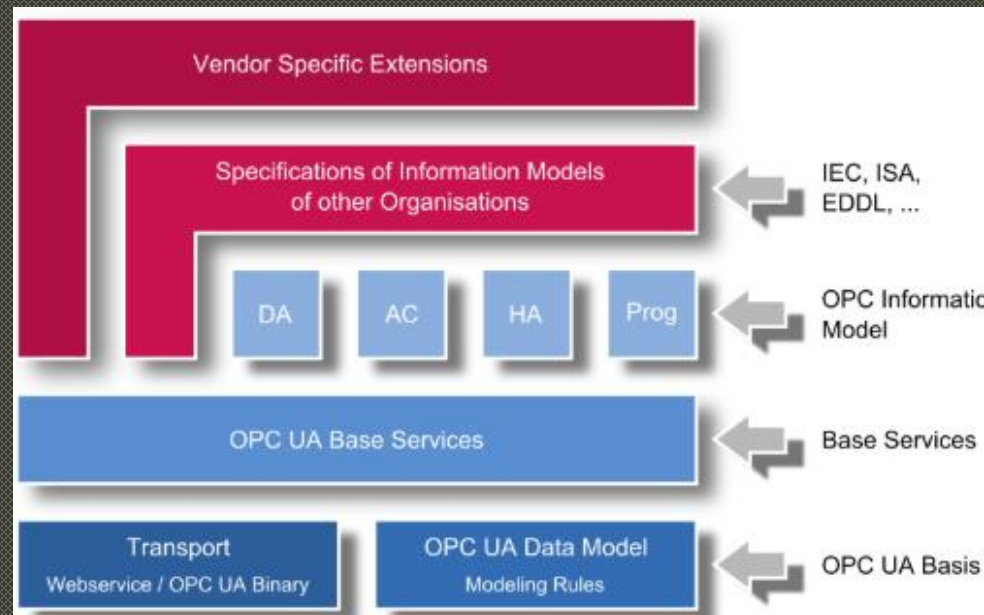
- Data modeling sets the rules and building blocks to represent any kind of device, system, or process using OPC UA.
- It defines how to structure the address space (where all information is stored and accessed) and provides basic types to create a type hierarchy (like templates or categories).
- These basic elements can be extended by industry-specific or vendor-specific information models.
- It also includes advanced features like state machines (used to model behavior and changes over time) to support more complex systems.



9B.3 OPC UA - System Architecture

- UA Services are the core communication functions in OPC UA.
 - Servers provide information (like sensor data or machine status), and clients (like HMIs or SCADA systems) request and use that information.
 - UA Services define how this exchange happens - they are the interface between clients and servers.
 - UA services are described in a general (abstract) way, so they are not tied to any specific hardware or software.
 - The actual movement of data (sending and receiving) happens through transport mechanisms like TCP or HTTP.

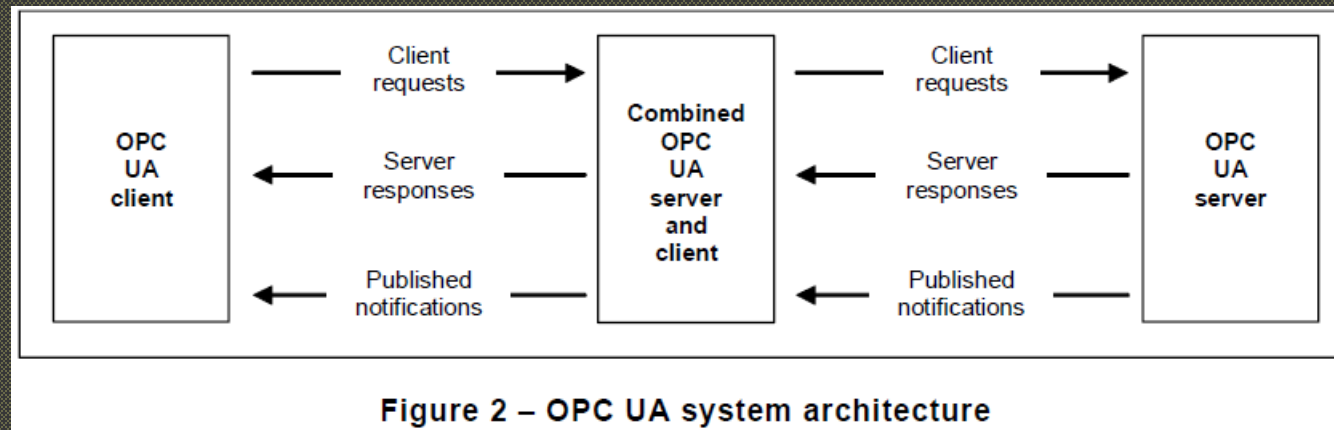
Data Access (DA),
Historical Data Access (HDA),
Alarms and Conditions (AC),
Electronic Device Description Language (EDDL)



9B.3 OPC UA - System Architecture

The OPC UA systems architecture views Clients and Servers as interactive partners.

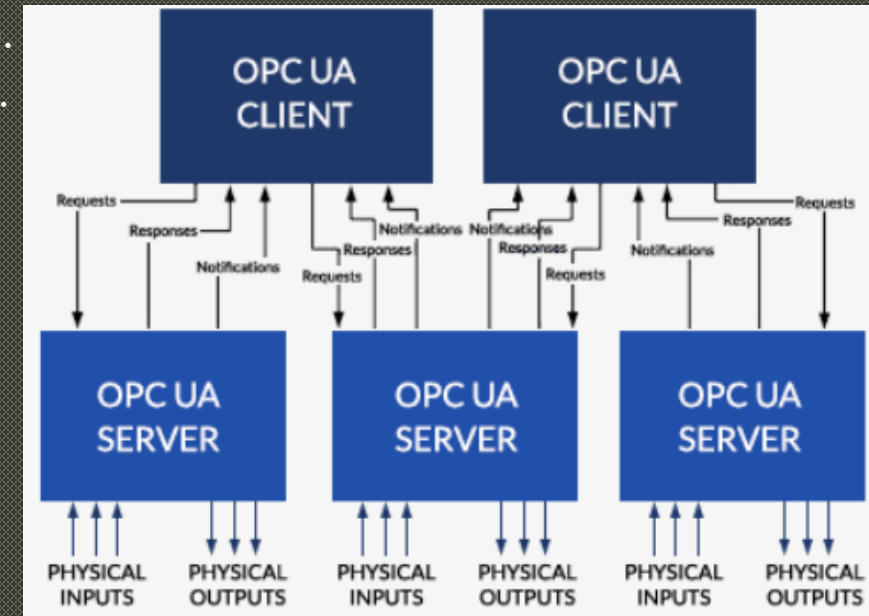
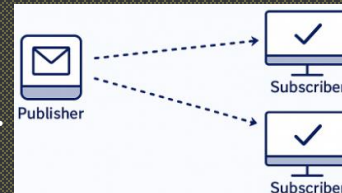
- A single system can host multiple Clients and Servers.
- Each Client can connect to and interact with one or more Servers at the same time, and each Server can handle interactions with multiple Clients simultaneously.



9B.3 OPC UA - System Architecture

OPC UA is a platform-independent communication standard that allows different systems and devices - from sensors to enterprise software - to exchange data securely across various networks. Two Communication Models:

- Client-Server Model:
 - A Client asks for data or services.
 - A Server provides them (like current or historical data, alarms, or events).
 - Servers describe their available data using a self-describing object model.
 - Clients can discover and interact with this data dynamically.
 - Communication is secure, robust, and uses both OPC UA-defined and vendor-specific data types.
 - Can run over various communication protocols like TCP, HTTP, or WebSockets.
- Publisher-Subscriber (PubSub) Model:
 - Instead of direct requests, a Publisher broadcasts data.
 - One or more Subscribers receive it.
 - Useful for fast, event-driven data transfer, such as real-time machine communication.



https://www.rtautomation.com/technologies/opcua/?srsltid=AfmBOoq6h6eSVDASa-P_fQq3WCXrxEOtxoqls6ZZR19KL Tnzk VDM5raz

9B.3 OPC UA - System Architecture

- OPC UA Interoperability and Flexibility
 - OPC UA servers don't have to implement every feature - instead, they support *Profiles*, which are predefined sets of features. This allows clients to easily connect based on what the server supports.
 - Standardized and discovery mechanisms ensure that different systems can communicate reliably across vendors.
- Seamless Migration from OPC Classic
 - OPC UA is designed to support smooth migration from older OPC systems based on Microsoft COM/DCOM technologies, including OPC DA (Data Access), HDA (Historical Data Access), and A&E (Alarms & Events).
- PubSub (Publish/Subscribe) Use Cases
 - Enables peer-to-peer communication between devices like controllers and HMIs without needing direct connections.
 - Supports event-driven workflows using message queues.
 - Facilitates simultaneous data logging to multiple systems.
 - Streams data to cloud platforms for advanced analytics and predictive maintenance.
 - Compatible with common messaging protocols such as AMQP, MQTT, and UDP.

AMQP - Advanced Message Queuing Protocol

UDP – User Datagram Protocol

MQTT - Message Queue Telemetry Transport

9B.3 OPC UA - System Architecture

An OPC UA Server is a software application that provides data to clients in a standardized, secure, and structured way. Think of it as:

- A librarian who organizes, protects, and gives access to important books (data).
- Clients (like HMIs, SCADA, or MES) are readers who come to ask for books (data or services).

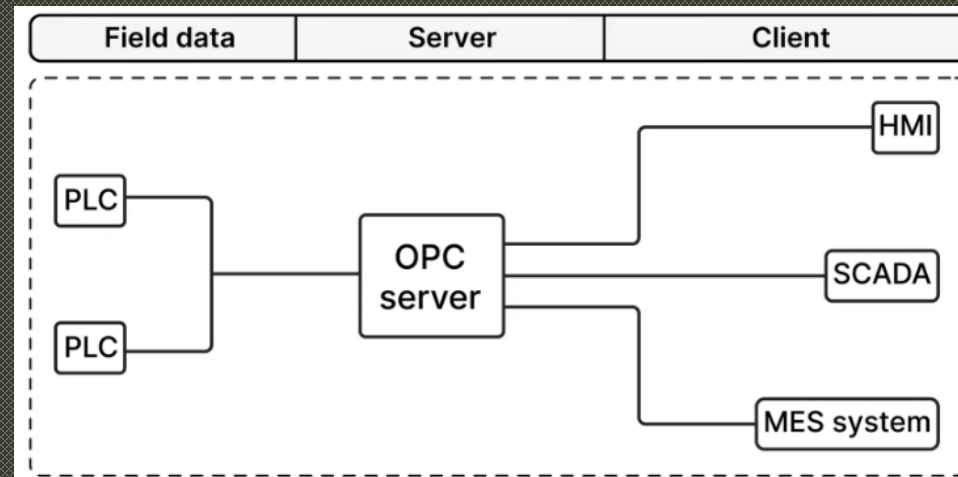
An OPC UA Server

1. Collects and Organizes Data

It connects to real devices (like PLCs, sensors, or databases) and organizes their data into a structured format called the Address Space.

For example:

- PLC input/output data
- Machine temperature, speed, alarms
- Sensor readings
- Historical data logs



9B.3 OPC UA - System Architecture

2. Exposes the Address Space

The Address Space is like a tree or folder system that organizes:

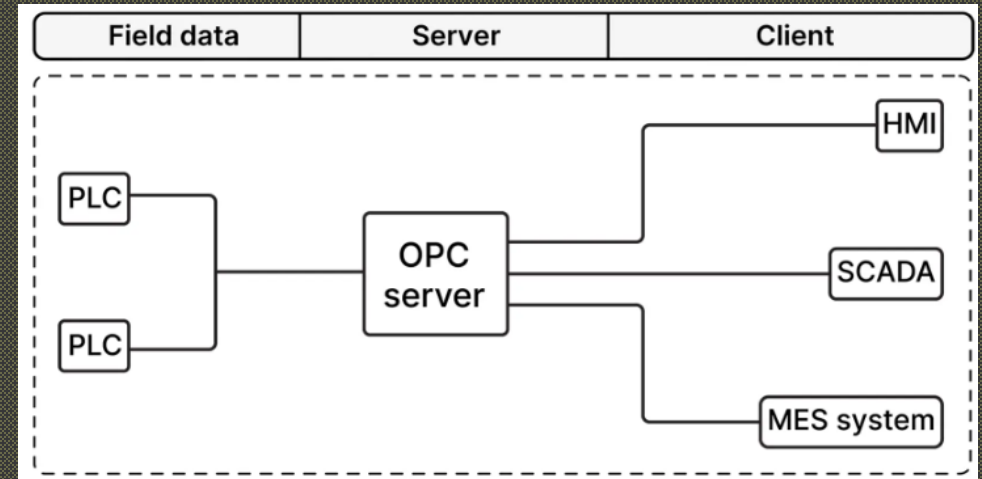
- Objects (e.g., devices)
- Variables (e.g., temperature, speed)
- Methods (actions like reset or calibrate)
- Events (alarms, notifications)

Clients can browse this structure and access what they need.

3. Handles Communication

The server uses OPC UA protocol to:

- Accept connections from Clients
- Respond to requests (read, write, subscribe)
- Send data in real time
- Notify about events or alarms
- Allow historical data queries



<https://www.realpars.com/blog/opc-ua-basics>

4. Manages Security

Security in OPC UA is a core feature:

- Encryption: Keeps data private
- Authentication: Verifies who is connecting
- Authorization: Grants permissions based on user roles
- Auditing: Keeps logs of what happened and who did what

9B.3 OPC UA - System Architecture

5. Supports Interoperability

OPC UA Servers follow a standard defined by the OPC Foundation.

This means:

- Clients from different vendors (like Siemens, Rockwell, Schneider) can still work together.
- No need to write custom drivers for each brand of device.

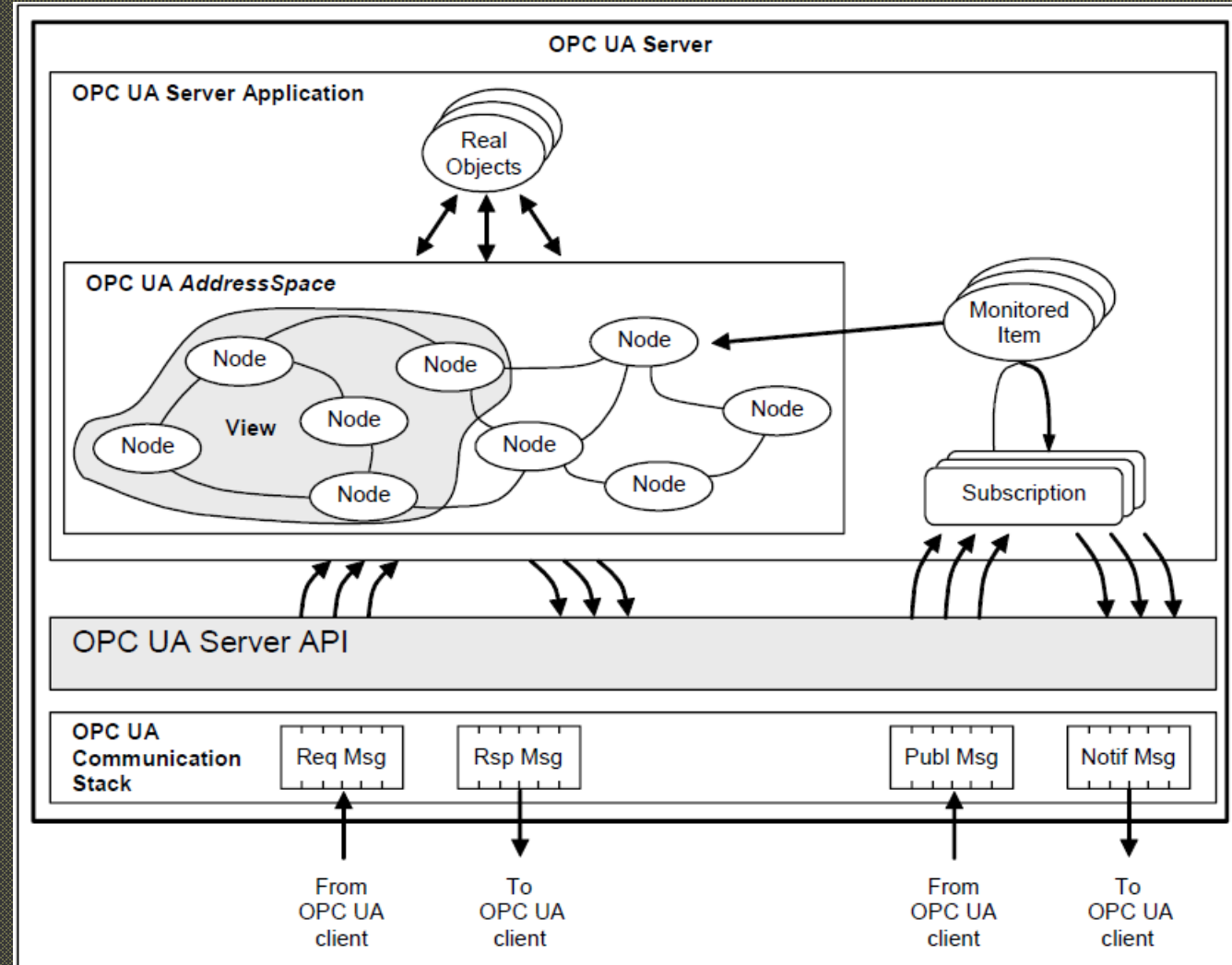


Figure 4 – OPC UA Server architecture

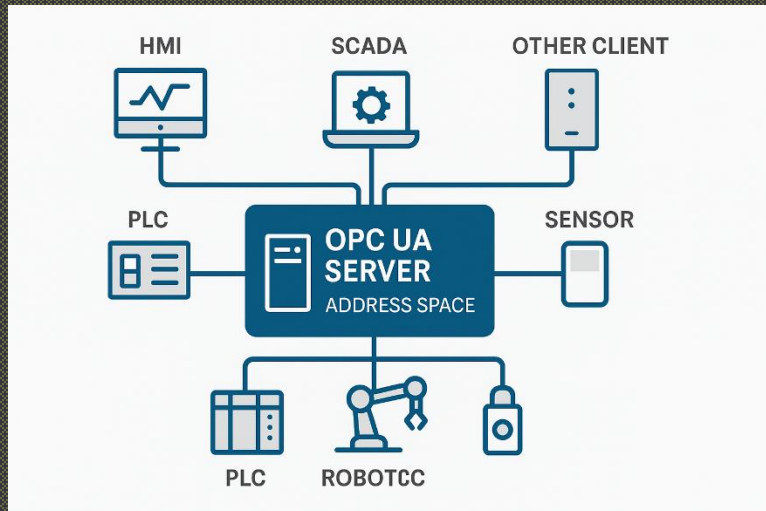
API - for Application Programming Interface

9B.3 OPC UA - System Architecture

Real-Life Analogy

Imagine a hotel front desk (the OPC UA Server):

- It knows all the rooms (devices)
- It can tell you who is in which room (variables)
- It can wake someone up (call a method)
- It has rules for who can access what (security)
- It keeps logs of all check-ins/check-outs (auditing)



Example in Industrial Automation

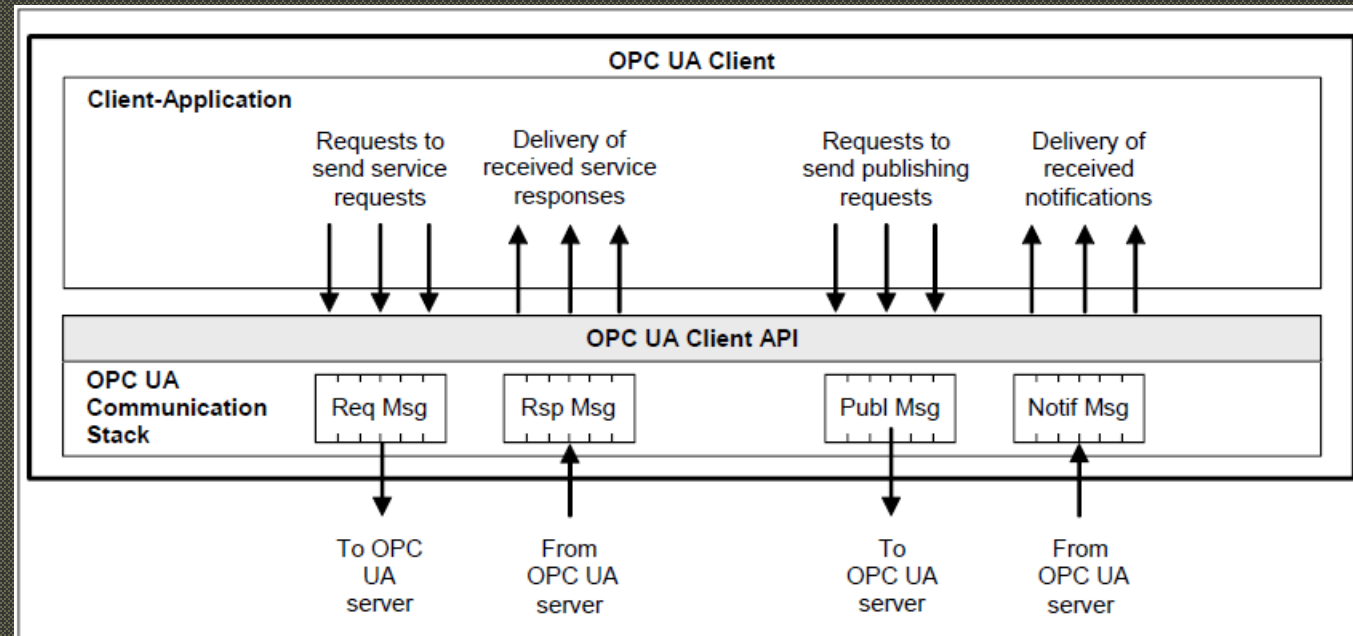
Scenario: We are monitoring a packaging machine with a PLC.

The OPC UA Server:

- Reads the PLC's data (like current speed, number of boxes packed, motor status)
- Organizes it in an Address Space
- Allows a SCADA Client to:
 - Read live values
 - Receive an alarm when the motor overheats
 - Call a method to reset a counter
 - Log historical performance data
- Ensures only authorized users can reset or stop the machine

9B.3 OPC UA - System Architecture

- Client Application - What you want to do?
 - This is the part that says, “I want to read data,” or “I want to check if an alarm is active.”
 - It is the main brain of the Client - deciding what needs to be done.
- Client API - The translator
 - The Client does not speak directly to the network. Instead, it uses the API - a set of tools that helps turn those ideas (“read temperature,” “write value”) into a format that can be understood by the next part.
- Communication Stack - The delivery service
 - This part takes the translated messages and sends them over the network to the Server.
 - It also receives responses and brings them back to the Client Application - like a delivery driver picking up and dropping off packages.



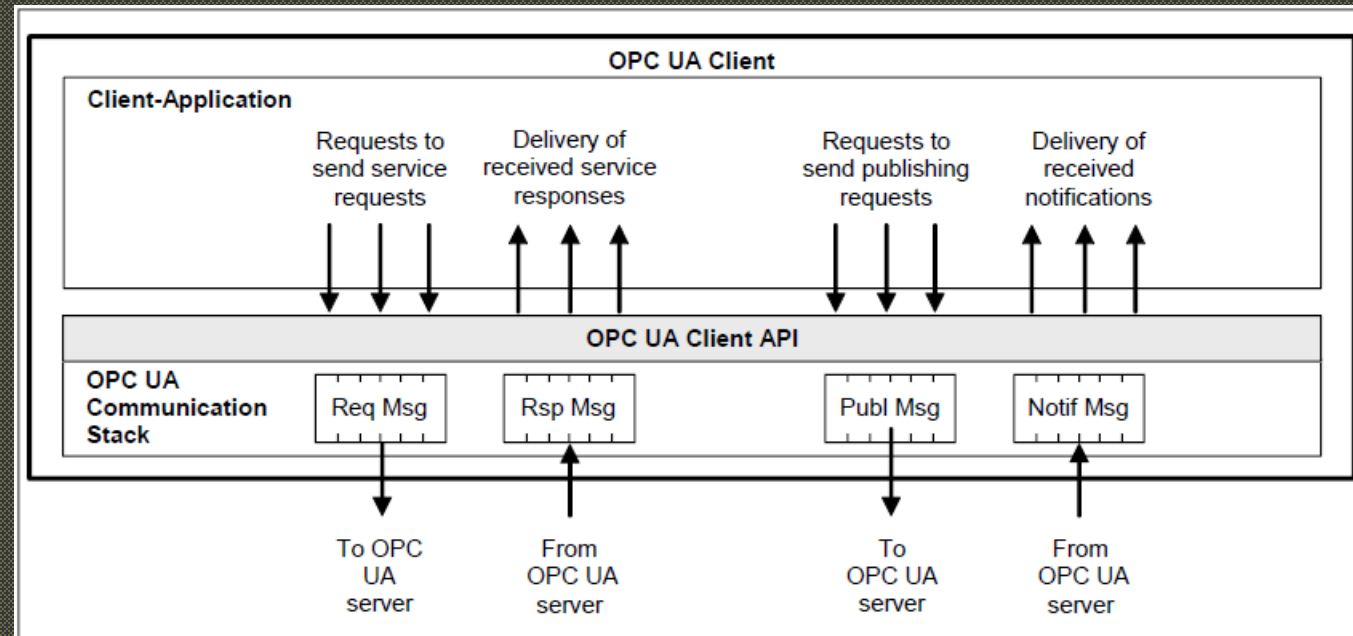
API - for Application Programming Interface

9B.3 OPC UA - System Architecture

The Client is the one asking questions, and it uses a translator (API) and a messenger (Communication Stack) to get answers from the Server

- The Client Application says: “Get me the temperature from the PLC.”
- The API translates that request into a message format.
- The Communication Stack sends that message over the network to the Server.
- The Server replies with the temperature.
- The Communication Stack receives it, and the API hands it back to the Client Application.

This setup makes the OPC UA Client flexible, easy to manage, and capable of working across many different devices and systems.

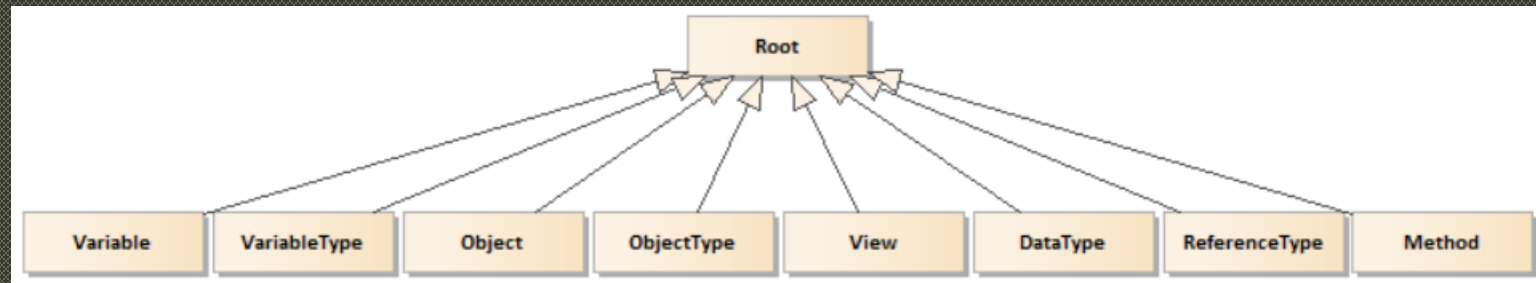


9B.4 OPC UA - Address Space and Information Models

- Unified Address Space:
 - Combines real-time data, alarms/events, and historical data into one logical structure.
 - Integrated Services: A consistent set of services for browsing, reading/writing data, and managing security.
 - Transports: OPC UA TCP, HTTPS, WebSockets
 - Multiple Data Formats: Supports binary, XML, and JSON; Clients can auto-detect and use data at runtime.
 - Encodings: UA Binary (fast), XML (structured), JSON (cloud-friendly)
 - Flexible Data Models: Servers can expose multiple hierarchies and object types, customized per application.
 - Built-in Security: Includes encryption, authentication, and session management.

OPC UA TCP is a communication protocol that forms the foundation for secure and reliable data exchange in industrial automation.

WebSockets provide a transport protocol for exchanging OPC UA messages over a standard web server connection (between clients and servers).



24

9B.4 OPC UA - Address Space and Information Models

- Address Space defines how information is structured, organized, and accessed within an OPC UA server.
- It serves as a virtual representation of the system, exposing data and services in a standardized, object-oriented way that clients can interact with - dynamically and securely.
- Address Space is a collection of Nodes interconnected by References, forming a flexible and extensible information model.
- It represents all elements of the automation system - including devices, sensors, variables, alarms, events, methods, historical data, and their relationships - in a browsable and well-structured hierarchy.

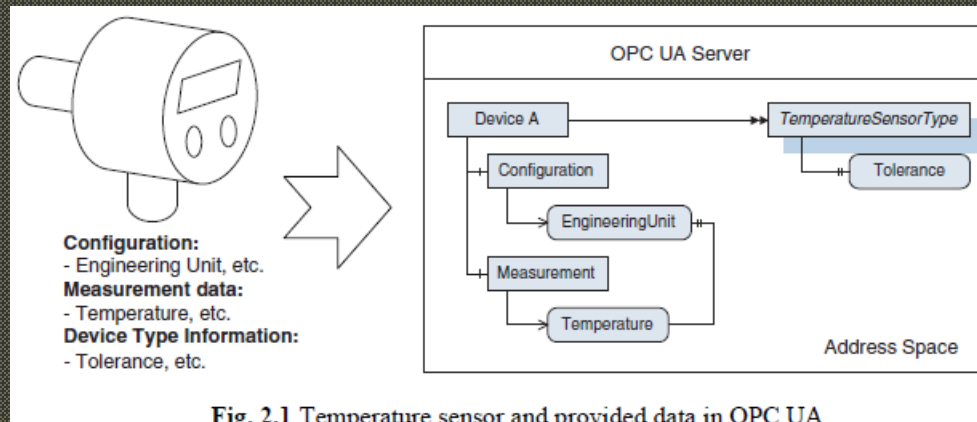
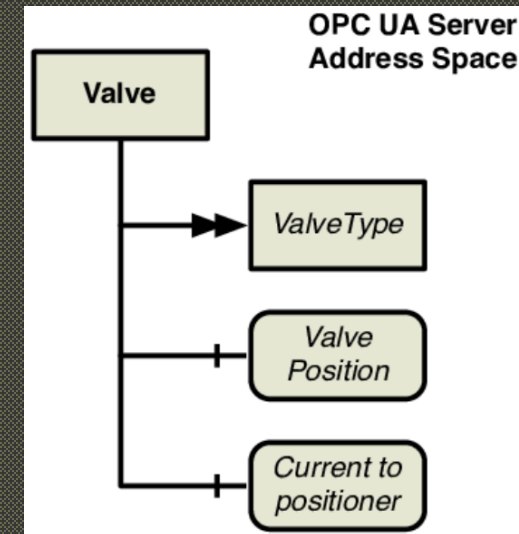
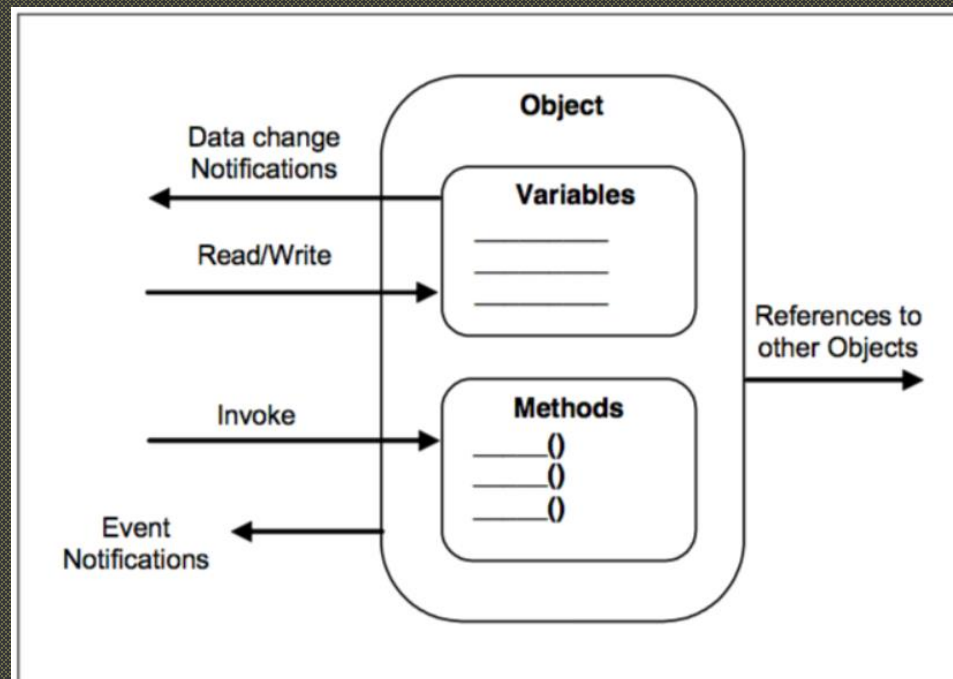


Fig. 2.1 Temperature sensor and provided data in OPC UA



9B.4 OPC UA - Address Space and Information Models

- The primary goal of the OPC UA Address Space is to offer a standardized way for servers to present objects to clients.
- The OPC UA Object Model supports this by defining objects through variables and methods, and it enables relationships between objects.
- These elements are represented as nodes in the Address Space, each assigned to a Node Class that reflects a specific part of the Object Model.

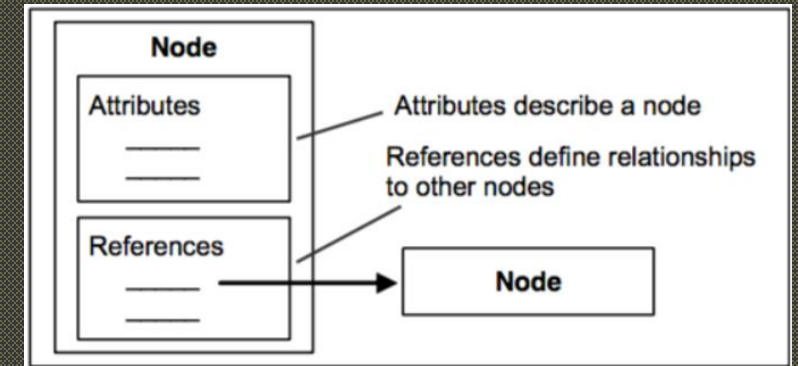


9B.4 OPC UA - Address Space and Information Models

Main Building Blocks of the Address Space

- Nodes:
 - A Node is an entity in the server that holds data or describes system functionality. There are several standardized Node classes:

Node Class	Description
Object	Represents real-world entities (e.g., machines, sensors)
Variable	Holds dynamic values (e.g., temperature, pressure)
Method	Represents executable functions (e.g., reset, start)
ObjectType	Defines templates for Objects
VariableType	Defines templates for Variables
ReferenceType	Defines the nature of relationships
DataType	Defines custom or standard data structures(e.g., Double, Boolean)
View	Creates filtered views of the Address Space



<https://qiuyuqi.gitbooks.io/opc-ua/content/Part3/Chapter4.html>

ObjectType: Defines a Template for all devices (e.g., sensors and actuators).

- Contains Variables like Serial Number, Manufacturer, and Status.

A VariableType defines the structure and behavior of a Variable, such as data type, access level, or value rank. Example: AnalogItemType:

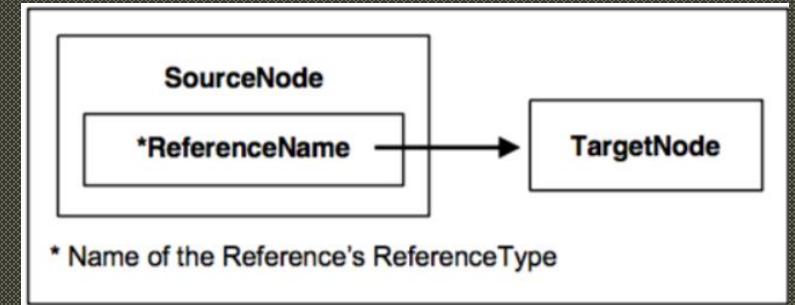
- A VariableType for analog values (e.g., a float or double).
- Includes properties like EngineeringUnits, EURange, and InstrumentRange.

9B.4 OPC UA - Address Space and Information Models

Main Building Blocks of the Address Space

- References:
 - Nodes are linked by References that define the type of relationship between them (e.g., HasComponent, Organizes, HasTypeDefinition). These create a graph-based structure, not limited to tree hierarchies.

Feature	Description
Structured	Models complex systems using an object-oriented approach
Dynamic	Clients can browse, explore, and discover Node information at runtime
Secure	Each Node can enforce user-based access control and audit logging
Semantic	Includes metadata like engineering units, descriptions, and data types
Multi-functional	Integrates real-time data, historical values, events, alarms, and methods in the same structure
Interoperable	Allows any OPC UA-compliant client to interpret and use the exposed data



<https://qiyuqi.gitbooks.io/opc-ua/content/Part3/Chapter4.html>

A ReferenceType defines relationships between Nodes.

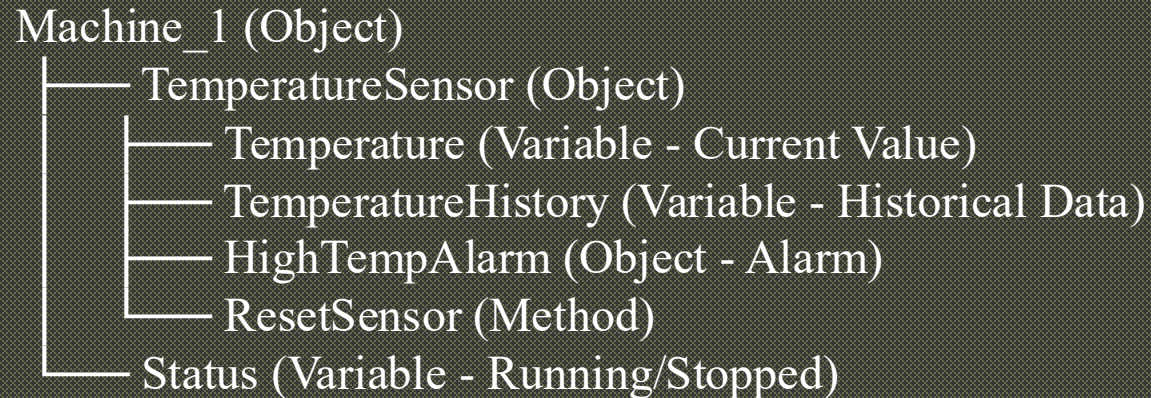
Example:

- Organizes:
 - Used to logically group Nodes (e.g., folders or views).
 - Example: Folder1 Organizes TemperatureSensor1
- HasComponent:
 - Shows composition, such as one object being part of another.
 - Example: Machine1 HasComponent Motor1

9B.4 OPC UA - Address Space and Information Models

Example of the Address Space

- Let's say you have a machine with a temperature sensor:



Each of these entries is a Node, linked through References, and can be discovered and accessed by OPC UA Clients (like SCADA, HMI, or IIoT gateways).

9B.4 OPC UA - Address Space and Information Models

Conceptual Diagram of the Address Space

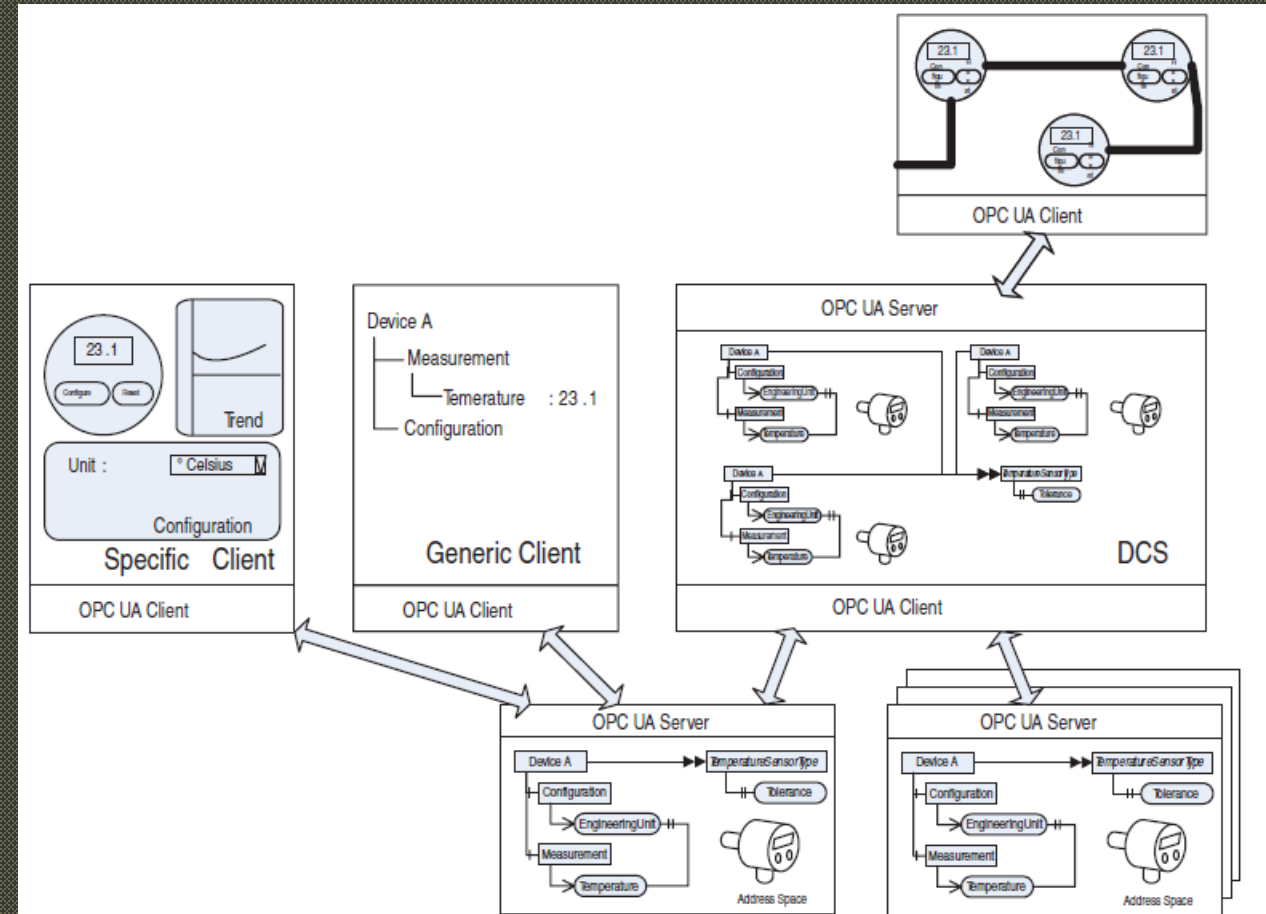
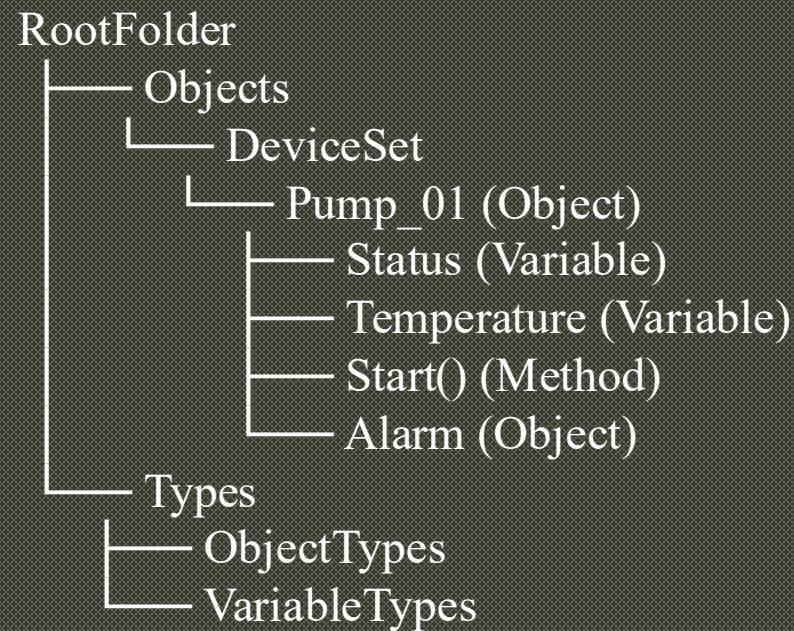


Fig. 2.2 Different scenarios how to access device data with OPC UA

This layout helps clients understand what exists, how it relates, and how to interact with it - all without needing to know the server's internal structure in advance.

9B.4 OPC UA - Address Space and Information Models

- In traditional systems, data is often arranged in a single, fixed hierarchy (like a tree with one path). OPC UA allows nodes (data elements) to have many kinds of relationships, not just parent-child. This means data can be structured in multiple, meaningful ways.
- Because of this flexibility, servers can organize and present the same data differently depending on the needs of different clients. For example, one client might need a functional view of a system (e.g., by machine function), while another might want a geographic view (e.g., by location).
- OPC UA uses type definitions to describe what kind of data is being accessed (e.g., temperature sensors, motors, alarms). This makes it easier for clients to understand and interact with unfamiliar systems.
- These features make OPC UA suitable beyond just industrial automation (like SCADA, PLCs, DCS). It can also be used at higher levels - for integrating enterprise systems, cloud services, or IT/OT convergence - where different types of systems must interoperate.

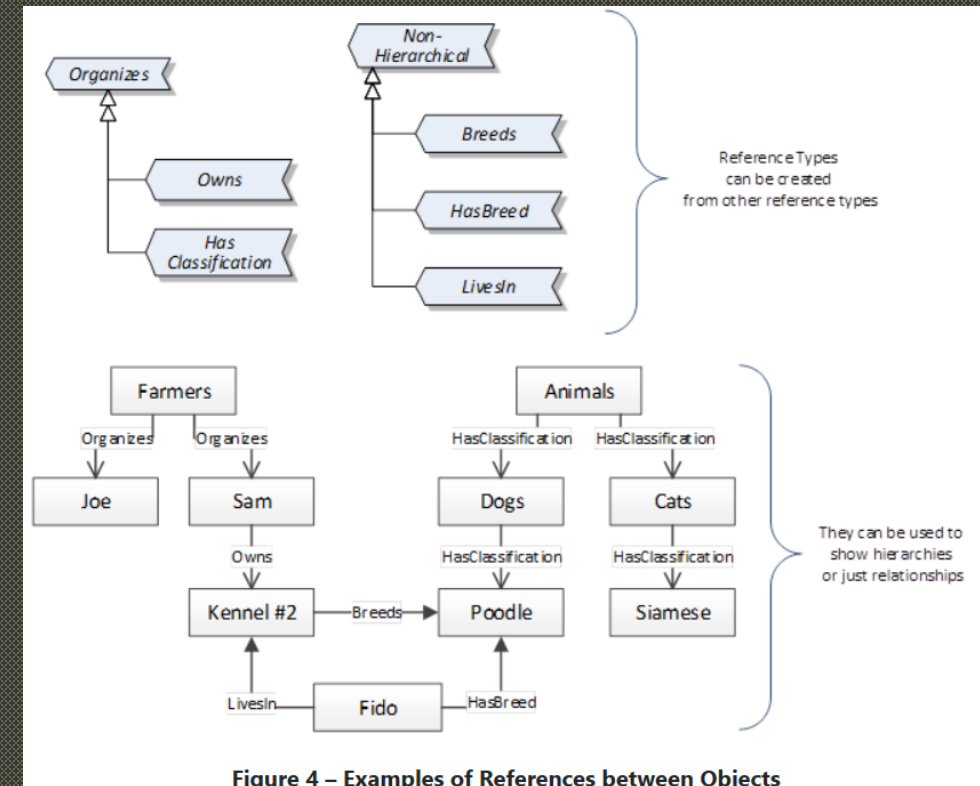


Figure 4 – Examples of References between Objects

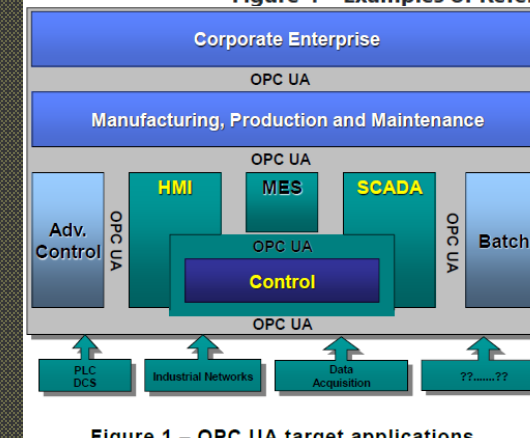


Figure 1 – OPC UA target applications

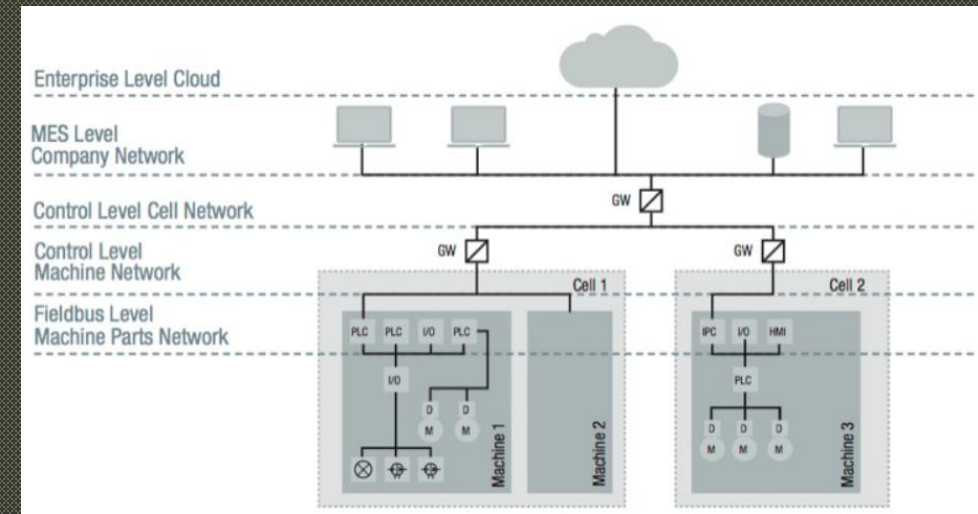
9B.5 OPC UA – Security Model

- High-level systems (like enterprise management) could be targets for sabotage.
- Low-level systems (like machines) could cause safety hazards or financial loss.
- OPC UA can be exposed to threats like malware on public or company networks.

- Different Environments, Different Risks:

- Some systems are tightly controlled and well-protected.
- Some are behind firewalls inside a company network.
- Some are exposed to open networks or harder-to-control users.
- Some are embedded in machines with no internet access at all.

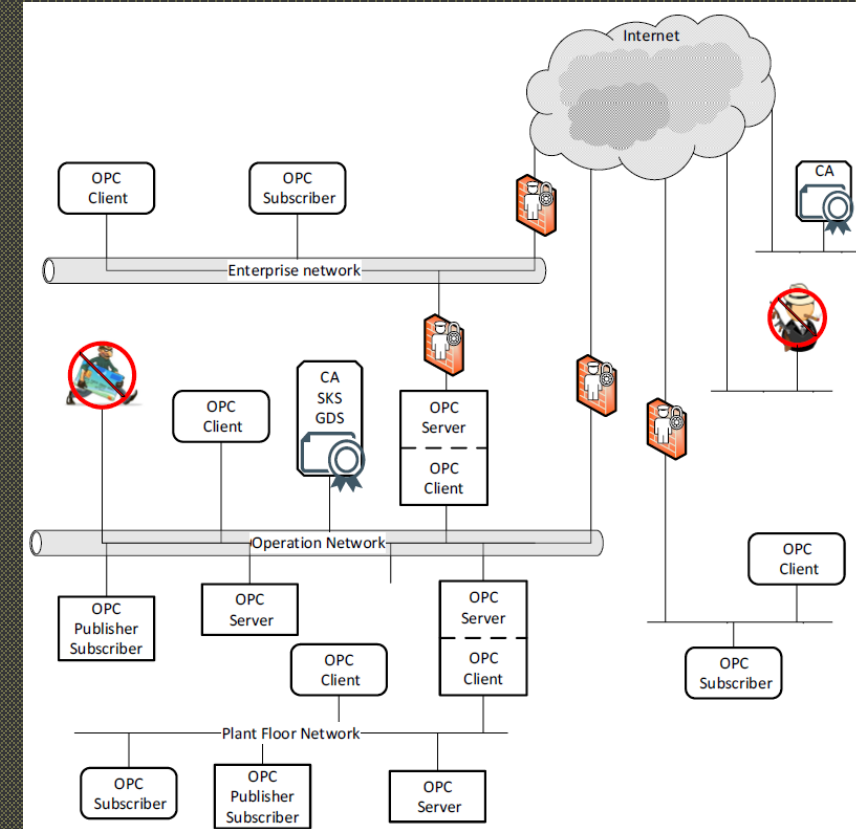
- OPC UA is designed to adapt to all these situations by offering:
 - A flexible security system that can be customized.
 - Support for multiple communication methods (like client-server or publisher-subscriber).
 - Global tools like:
 - Certificate Management (to identify devices/users)
 - Credential & Authorization Services (to manage access)
 - Global Discovery Server (to help devices find each other securely)



<https://opconnect.opcfoundation.org/2018/06/practical-security-guidelines-for-building-opc-ua-applications/>

9B.5 OPC UA – Security Model

- OPC UA ensures security through a multilayered approach and focuses on four main security goals:
 - Who is talking (Client/Server and user authentication),
 - What's being said (data integrity),
 - Who can see it (data confidentiality),
 - Proving what happened (traceable logs).
- 1. Certificate Management
 - Trust Lists: Both OPC UA servers and clients maintain trust lists with X.509 certificates of trusted entities.
 - Certificate Verification: Upon connection, the server checks the client's certificate against its trust list, while the client verifies the server's certificate.
 - Authentication: This verification process confirms the identity of both client and server, ensuring mutual trust.
 - Certificate Authorities (CAs): CAs can issue certificates to create trusted hierarchies, enhancing security.



Certificate Authority (CA)
Global Discovery Server (GDS)
Security Key Service (SKS)

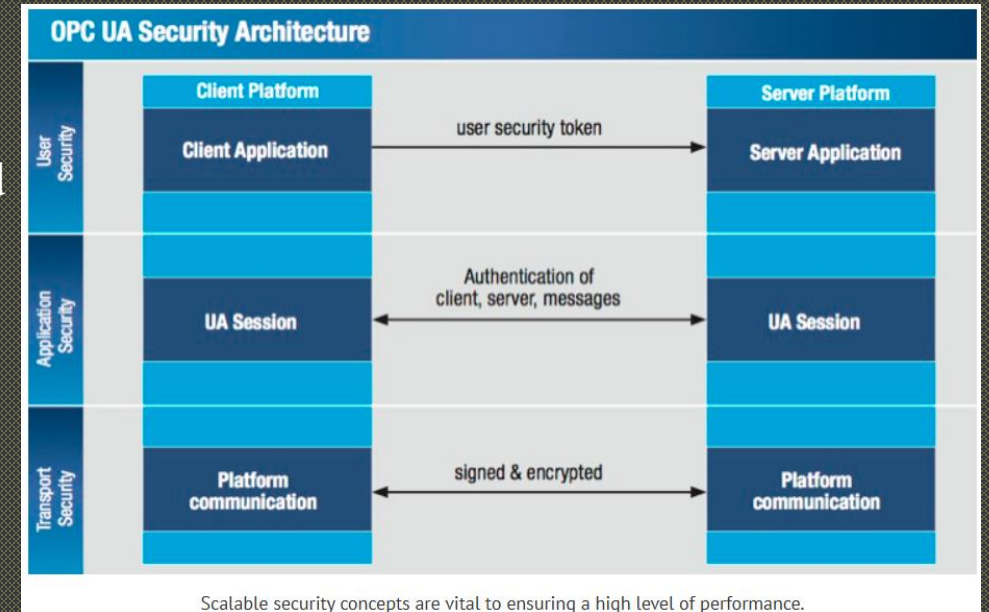
9B.5 OPC UA – Security Model

2. Secure Channel Establishment

- **OpenSecureChannel:** Clients initiate this service to request a secure channel from the server.
- **Encryption and Signatures:** Upon establishment, OPC UA employs encryption and digital signatures to ensure message confidentiality and integrity.
- **Key Exchange:** Cryptographic keys, derived from shared secrets between client and server, secure the communication.
- **Transport Level Security:** OPC UA utilizes underlying transport protocols (e.g., TCP) to implement encryption and signatures.

3. User Authentication

- **User Identity Tokens:** OPC UA supports multiple authentication methods, including username/password, X.509 certificates, and JSON Web Tokens (JWT).
- **Authorization:** Following authentication, authorization mechanisms define the user's access rights to specific resources.



<https://opconnect.opcfoundation.org/2018/06/practical-security-guidelines-for-building-opc-ua-applications/>

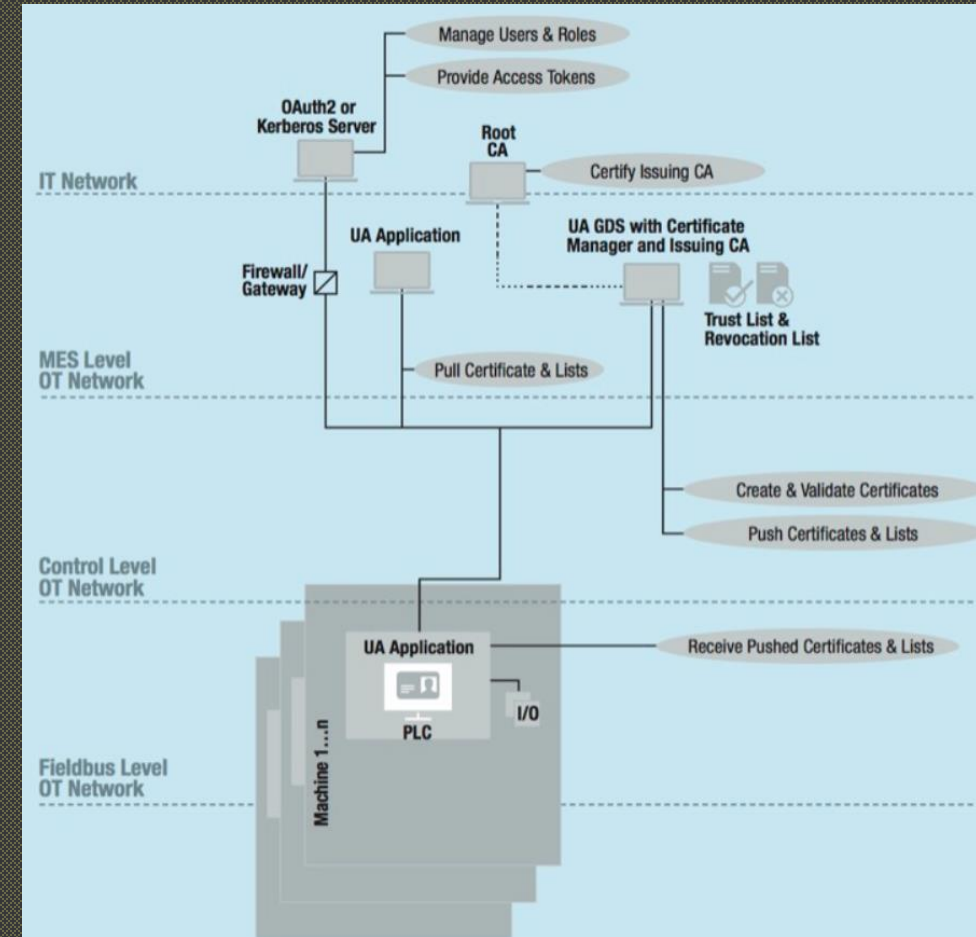
9B.5 OPC UA – Security Model

4. Security Profiles

- Standardized Security: OPC UA defines security profiles that outline the security mechanisms tailored to various communication scenarios.
- Compliance: Clients and servers must adhere to these specific security profiles to ensure both interoperability and robust security.

5. Additional Security Features:

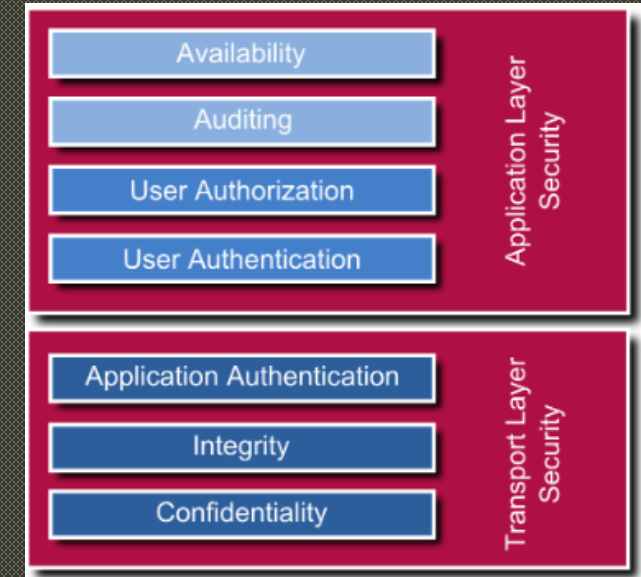
- Timeout Management: OPC UA implements configurable timeouts to prevent denial-of-service attacks.
- Error Detection: Mechanisms for detecting and handling communication errors are also in place.
- Communication Failure Recovery: OPC UA provides functionalities to recover from communication disruptions.



9B.5 OPC UA – Security Model

Tracking What Happens

- OPC UA supports auditing, meaning you can track who did what and when.
- Audit logs can be matched between Client and Server to trace issues.
- Servers can also send Event Notifications about important actions, which Clients can record.
- OPC UA defines standard audit parameters.
- Supported features depend on the chosen Profile.



<https://www.ascolab.com/en/technology-unified-architecture/technology-security.html>

Category	Example Standard Parameter	Description
Data Types	Boolean, Int32, Double	Common data formats used in communication.
Security	SecurityPolicy#Basic256Sha256	A defined set of encryption/signing rules.
Session Settings	SessionTimeout, MaxRequestMessageSize	Parameters that define session behavior.
User Identity Tokens	UserNameIdentityToken, X509CertificateToken	Standard ways users can authenticate.
Node Attributes	NodeId, BrowseName, DataType, AccessLevel	Standard information used to describe a data point or object.

A Profile is a formal specification that groups together related features, services, data models, or security mechanisms which an OPC UA application (Client or Server) may implement. For example: Data Access Profile, and use Basic256Sha256 encryption.

9B.5 OPC UA – Security Model

In the Client/Server setup:

- Communication happens in sessions between a client (e.g., HMI or SCADA) and a server (e.g., PLC or data source). Security is layered across:
 - Application Layer: Handles user authentication, authorization, and audit logging.
 - Communication Layer: Ensures message integrity, encryption, and certificate handling.
 - Transport Layer: Handles the actual sending of data using secure protocols like UA-TCP or HTTPS.

For Publish-Subscribe (PubSub):

- Data is published by one device and received by many others (subscribers).
- It works differently than request-response models (like Client/Server).
- Security in PubSub is usually handled by:
 - Message signing and encryption at the message level.
 - Use of security tokens, certificates, and access control for trusted subscribers.

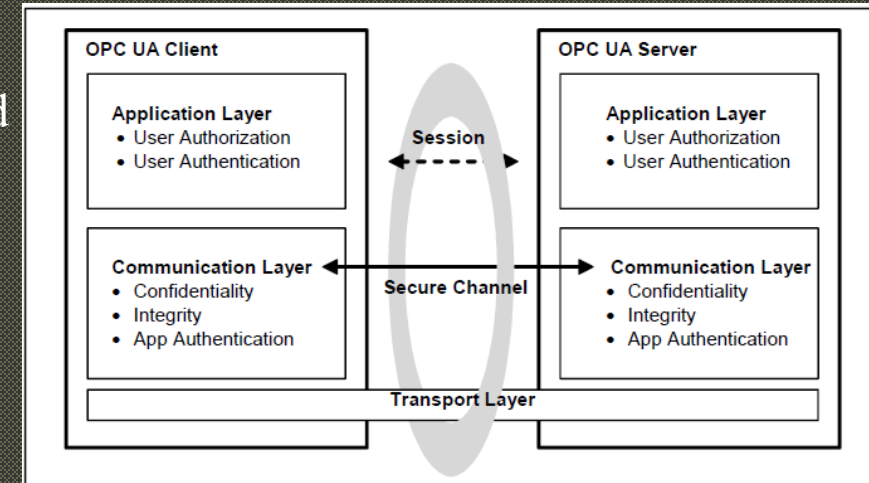


Figure 2 – OPC UA security architecture – Client / Server

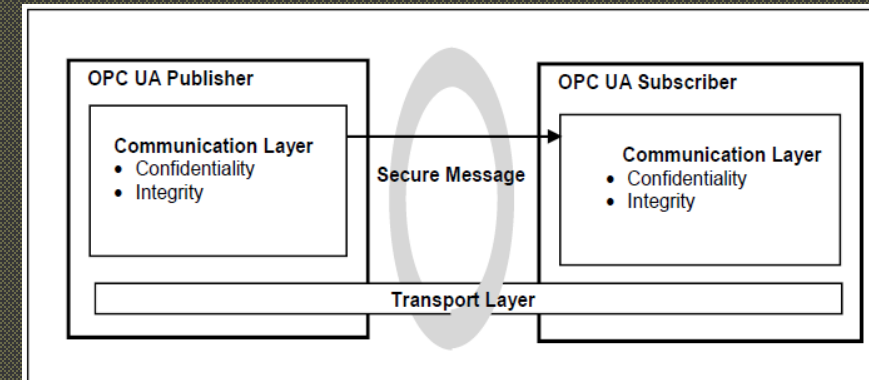


Figure 3 – OPC UA security architecture- Publisher - Subscriber

9B.5 OPC UA – Security Model

OPC UA's PubSub model allows devices (like controllers, HMIs, sensors, etc.) to publish and subscribe to data. This can happen in two ways:

1. With a Broker (like MQTT)
 - A Broker (like MQTT or AMQP) acts as an intermediary.
 - Publishers send data to the Broker; Subscribers get it from the Broker.
 - Security options:
 - Uses shared Symmetric Keys (like broker-less) for encryption & integrity.
 - The Broker can have its own rules for authorizing:
 - Publishers (who can send) & Subscribers (who can receive)
 - These rules depend on the middleware (e.g., MQTT, AMQP), not OPC UA.
2. Broker-less (devices talk directly to each other)
 - Devices talk directly to each other (no middleman).
 - Security is managed using shared Symmetric Keys (same key for encryption and decryption).
 - These keys are distributed by a Security Key Server (SKS).
 - SKS uses standard OPC UA Client/Server authentication to verify who gets the key.

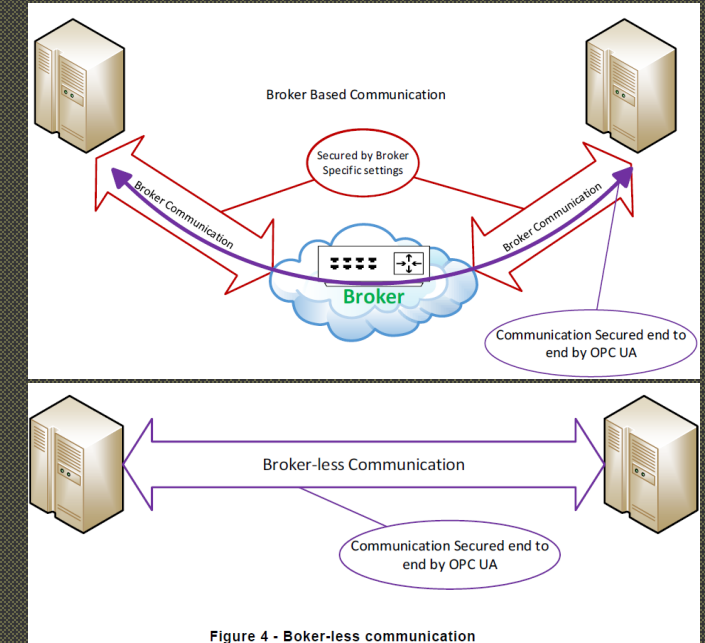


Figure 4 - Broker-less communication

Feature	Broker-less	Broker-based
Communication	Direct (device to device)	Via intermediary (Broker)
Encryption	Shared Symmetric Keys (SKS)	Shared Symmetric Keys (SKS) or Broker security
Authentication	Handled via SKS (Client/Server)	Middleware defines it
Risk	All devices in group share rights	Broker may see data if not encrypted
Security Configuration	OPC UA + SKS	OPC UA + Middleware + SKS

9B.6 OPC UA – Services

OPC UA Services are the building blocks that define how Clients (like HMIs, SCADA, or other software) interact with Servers (like PLCs, sensors, or data gateways).

Categories and Examples of OPC UA Services include:

1. Discovery Services

- These services help clients find OPC UA servers on a network, making it easy to connect without knowing their details upfront.
- Examples:
 - FindServers: Imagine you are at a party and want to find all your friends (servers). This service lets the client discover all available OPC UA servers on the network. For instance, a factory HMI might use this to locate a PLC server managing a conveyor belt.
 - GetEndpoints: This is like asking a friend for their contact info. It provides details (e.g., security settings, endpoints) about a specific server, so the client knows how to connect securely.
- Real-Life Analogy: It is like using a directory to find stores in a mall - Discovery Services help the client navigate the industrial "mall" of devices.

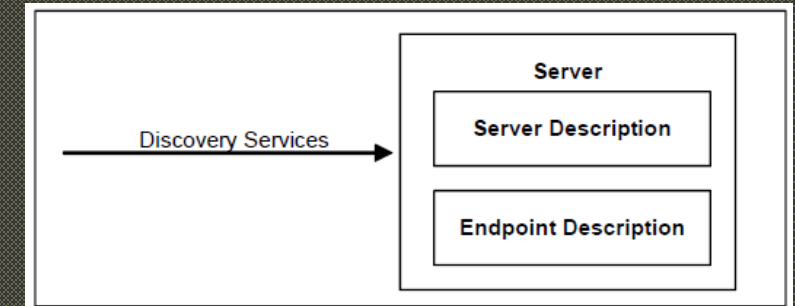


Figure 1 – Discovery Service Set

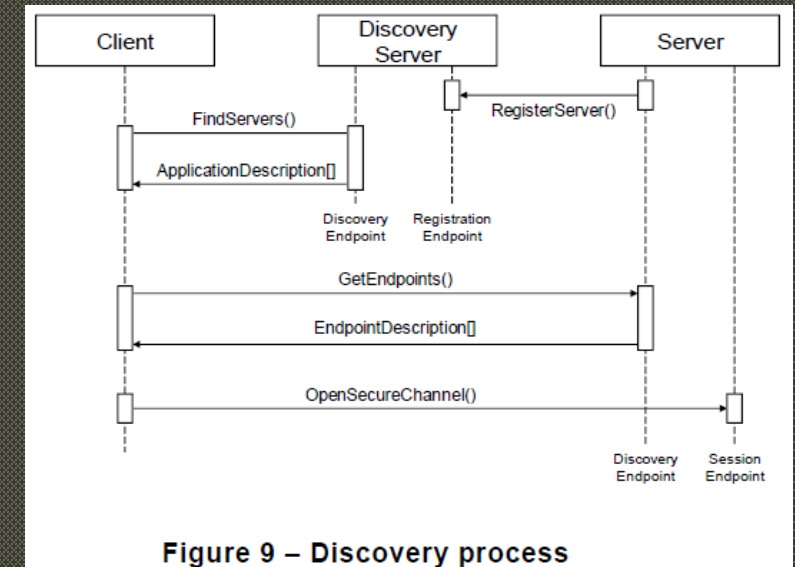


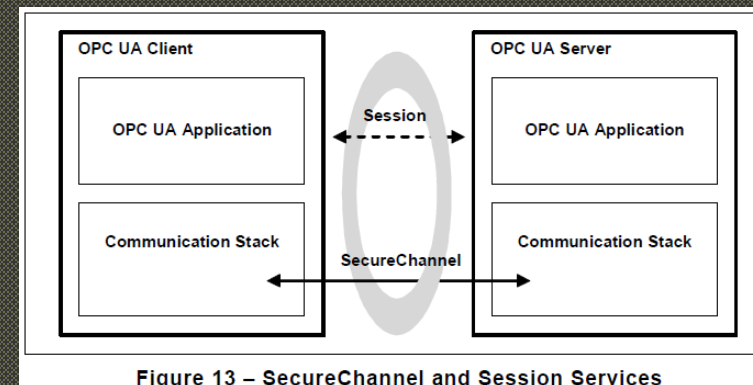
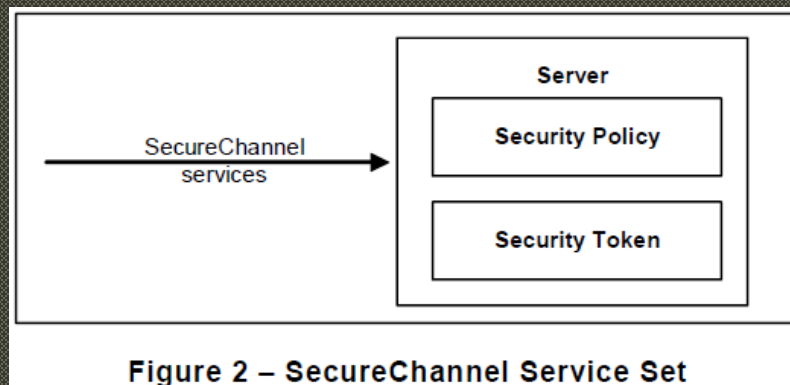
Figure 9 – Discovery process

9B.6 OPC UA – Services

Categories and Examples of OPC UA Services include:

2. Security Services

- These ensure that only authorized clients can access the server and that data is protected during communication.
- Examples:
 - OpenSecureChannel: This is like locking the door with a key. A client requests a secure connection, and the server sets up encryption (e.g., using Transport Layer Security - TLS) to protect data. For example, a remote operator might use this to safely control a pump.
 - CloseSecureChannel: This closes the secure connection when done, like locking the door after leaving. It ensures no one can sneak in after the session ends.
- Real-Life Analogy: Think of it as using a password and VPN to access your bank account online - Security Services keep industrial data safe.

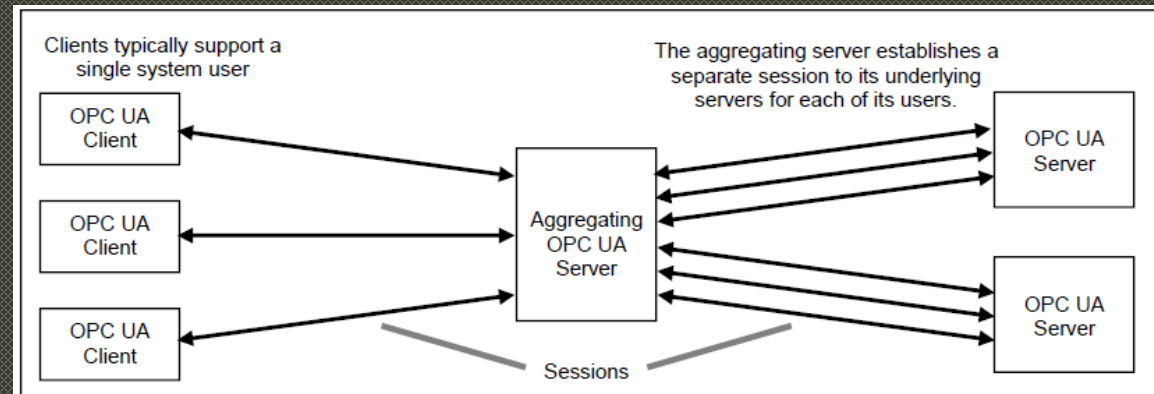
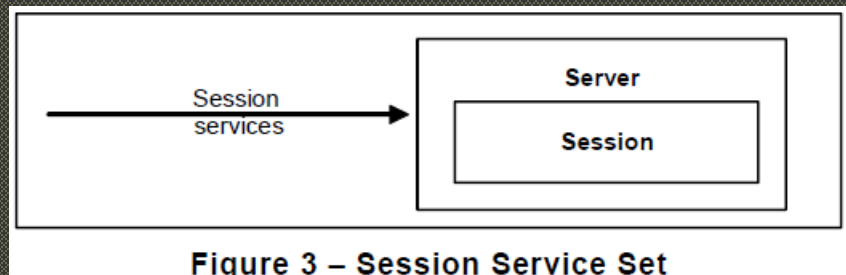


9B.6 OPC UA – Services

Categories and Examples of OPC UA Services include:

3. Session Services

- These manage the connection between a client and server, ensuring smooth and continuous interaction.
- Examples:
 - CreateSession: This is like starting a video call. The client opens a session with the server to begin communication, such as monitoring a motor's speed.
 - ActivateSession: This authenticates the user (e.g., with a username/password), like verifying your identity before the call starts. Only after this can the client access data.
 - CloseSession: This ends the session, like hanging up the call, ensuring resources are freed up.
- Real-Life Analogy: It is like logging into a meeting platform - Session Services keep the conversation active and secure.



9B.6 OPC UA – Services

Categories and Examples of OPC UA Services include:

4. Node Management Services

- These allow clients to browse, read, and write data in the server's Address Space.
- Examples:
 - Browse: This lets the client explore the Address Space, like looking through a catalog. For example, a client might browse to find a "Temperature" variable in a sensor.
 - Read: This retrieves data, like checking the catalog price. A client might read the current temperature (e.g., 25°C) from a thermostat.
 - Write: This updates data, like changing the catalog price. A client could write a new setpoint (e.g., 20°C) to adjust the thermostat.
- Real-Life Analogy: It is like browsing a library's catalog, checking out a book (Read), or returning it with notes (Write).

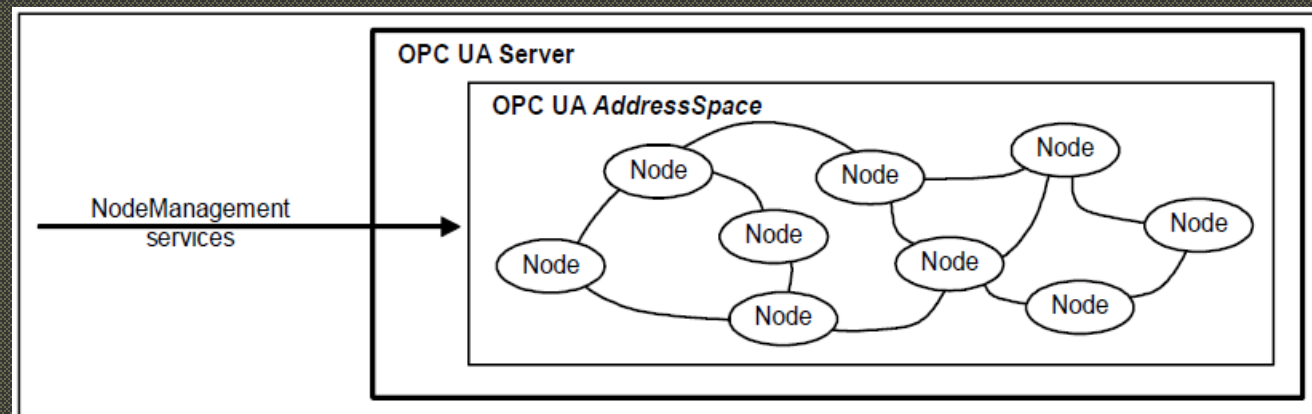


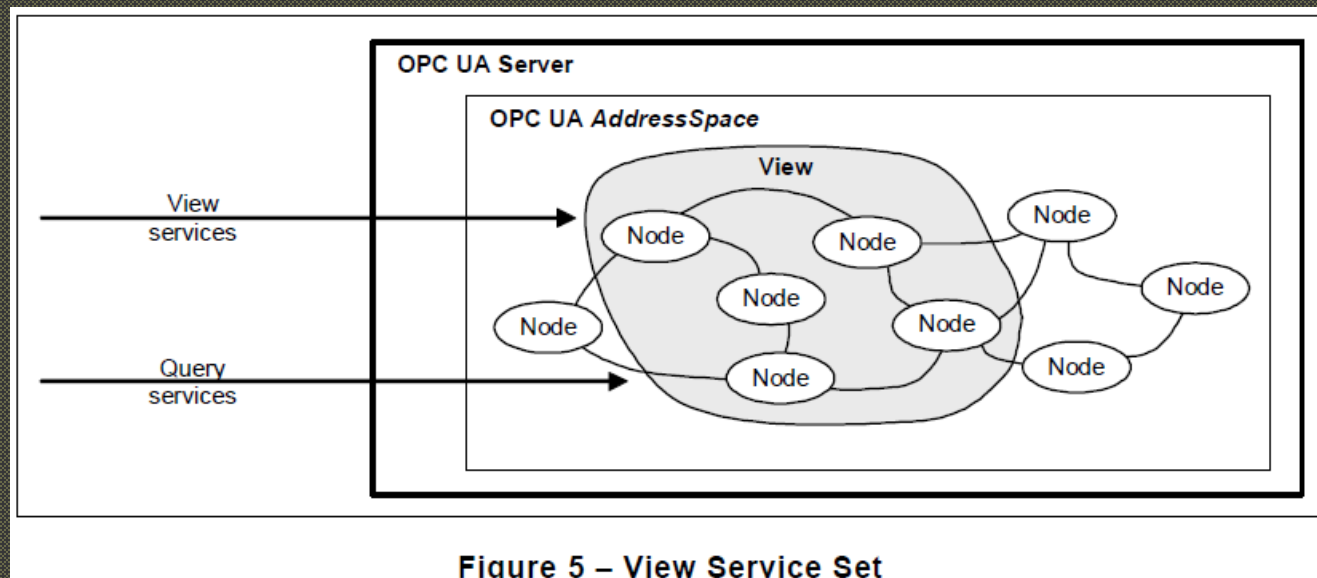
Figure 4 – NodeManagement Service Set

9B.6 OPC UA – Services

Categories and Examples of OPC UA Services include:

5. View Services

- These help clients filter and organize the Address Space to focus on relevant data.
- Examples:
 - BrowseNext: This continues browsing from a previous point, like turning the page in a book. A client might use it to see more sensor data after an initial browse.
 - TranslateBrowsePathsToNodeIds: This converts a path (e.g., "Factory/Motor1/Speed") into a NodeId, like finding a book's location code in a library.
- Real-Life Analogy: It is like using a library's search tool to narrow down books by category or location.

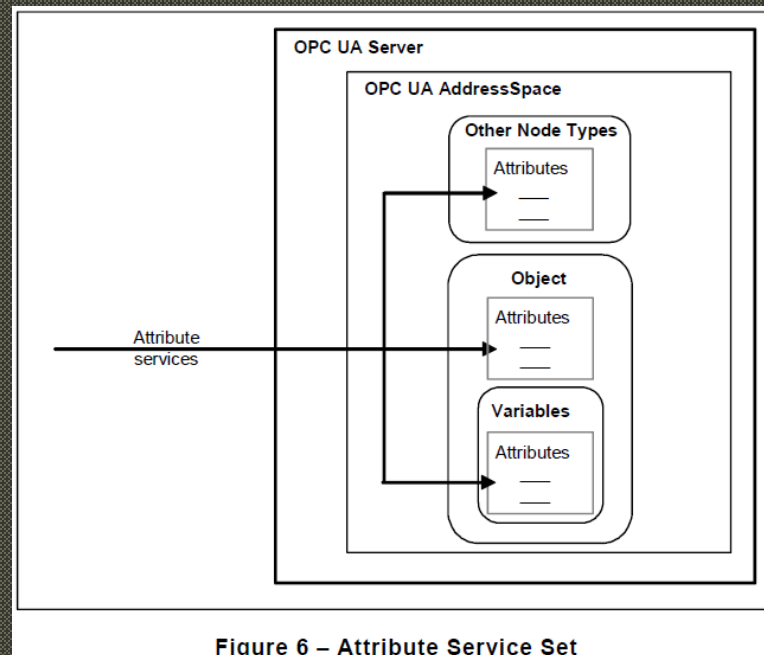


9B.6 OPC UA – Services

Categories and Examples of OPC UA Services include:

6. Attribute Services

- These provide detailed information about nodes in the Address Space.
- Examples:
 - Read: Gets specific attributes (e.g., value, timestamp) of a node, like checking a product's details online.
 - Write: Changes attributes, like updating a product's stock level.
- Real-Life Analogy: It is like looking at a product's label (Read) or editing its price tag (Write) in a store.

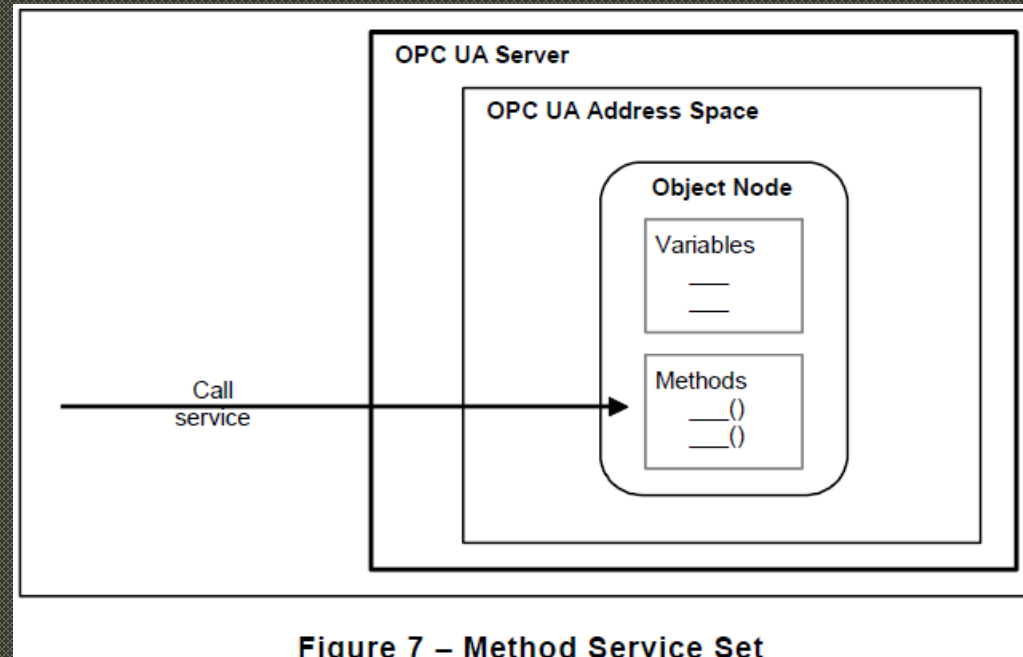


9B.6 OPC UA – Services

Categories and Examples of OPC UA Services include:

7. Method Services

- These allow clients to call functions or actions on the server.
- Examples:
 - Call: This executes a method, like pressing a button. A client might call a "StartMotor" method to turn on a machine.
- Real-Life Analogy: It is like asking a remote control to change the TV channel - Method Services trigger actions.

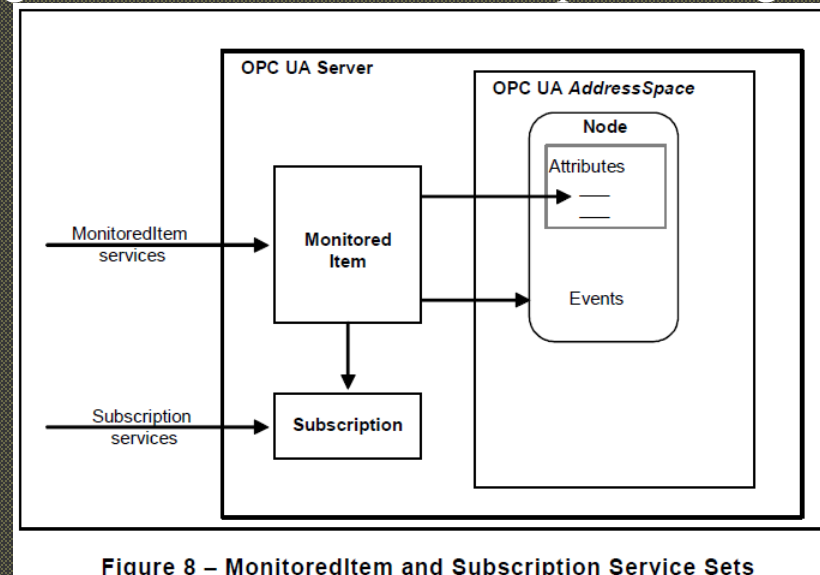


9B.6 OPC UA – Services

Categories and Examples of OPC UA Services include:

8. Subscription and Monitoring Services

- These enable real-time updates by notifying clients of changes without constant polling.
- Examples:
 - CreateSubscription: This sets up a subscription, like subscribing to a news alert. A client might subscribe to a pump's pressure changes.
 - CreateMonitoredItem: This monitors a specific item, like tracking a single news story. It could monitor a valve's status for updates.
 - Publish: This delivers notifications, like receiving the news alert. The server sends updates when the pressure changes.
- Real-Life Analogy: It is like getting text notifications for stock price changes instead of checking manually.

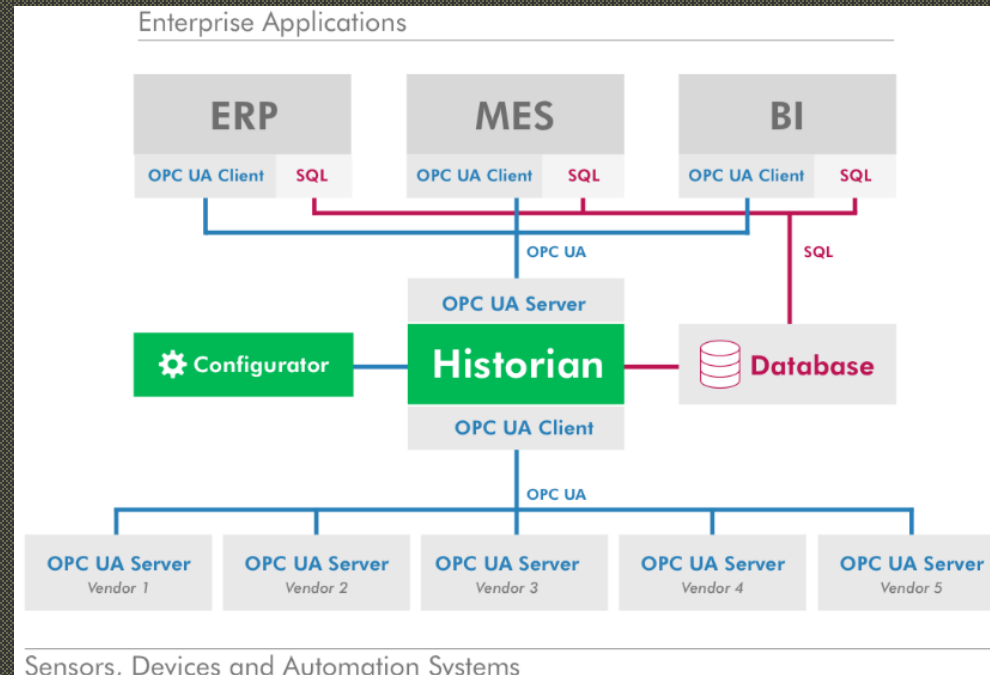


9B.6 OPC UA – Services

Categories and Examples of OPC UA Services include:

9. Historical Access Services

- These allow clients to retrieve past data for analysis.
- Examples:
 - HistoryRead: This fetches historical data, like reviewing last week's temperature logs from a sensor.
 - HistoryUpdate: This modifies historical data, like correcting a logged error (if permitted).
- Real-Life Analogy: It is like looking at your phone's weather history or editing a past entry in a diary.

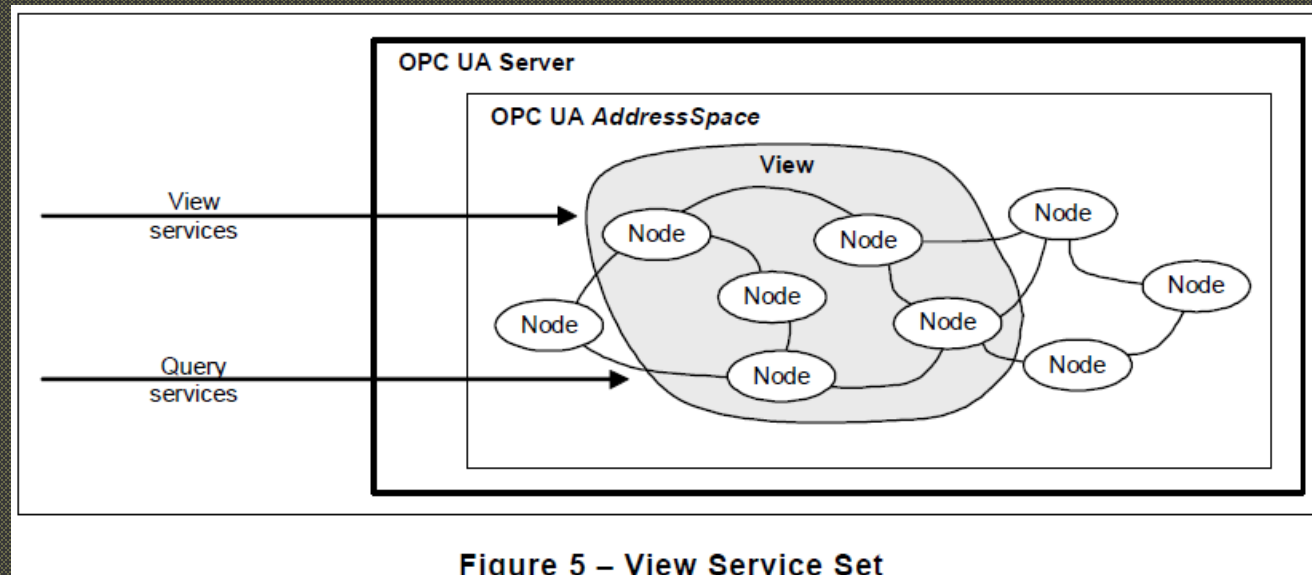


9B.6 OPC UA – Services

Categories and Examples of OPC UA Services include:

10. Query Services

- These enable complex searches within the Address Space.
- Examples:
 - QueryFirst: This starts a search, like looking for all motors in a factory.
 - QueryNext: This continues the search, like paging through results.
- Real-Life Analogy: It is like using a search engine to find specific items in a large online store.

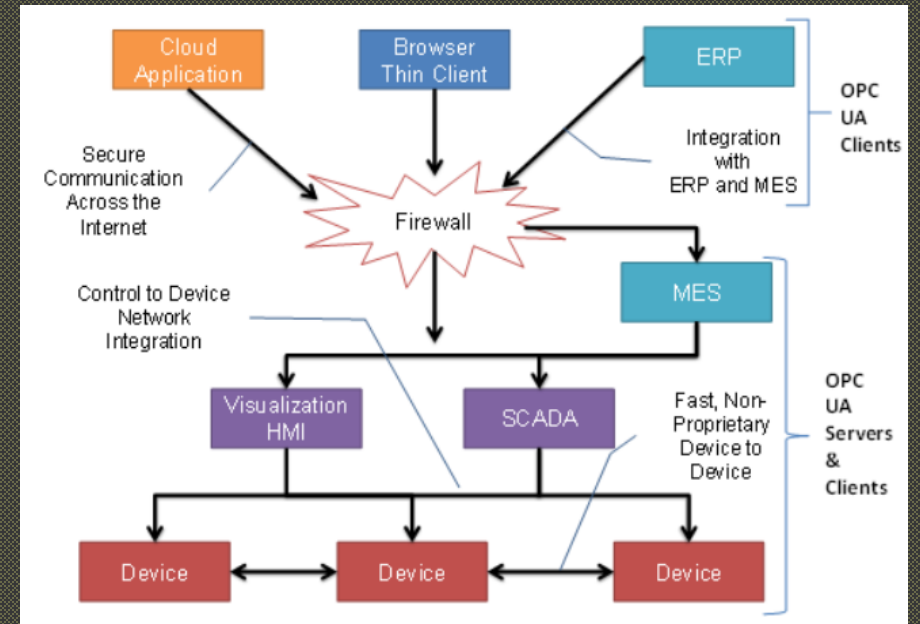


9B.6 OPC UA – Services

Categories and Examples of OPC UA Services - How It All Fits Together:

Imagine a smart factory where a client (e.g., an HMI) uses these services:

- Discovery: Finds the PLC server.
- Security: Sets up a secure channel.
- Session: Starts a session.
- Node Management: Reads the motor's speed.
- Subscription: Gets real-time updates if the speed changes.
- Historical Access: Reviews past performance.



<https://reference.opcfoundation.org/MachineTool/v100/docs/4.2>

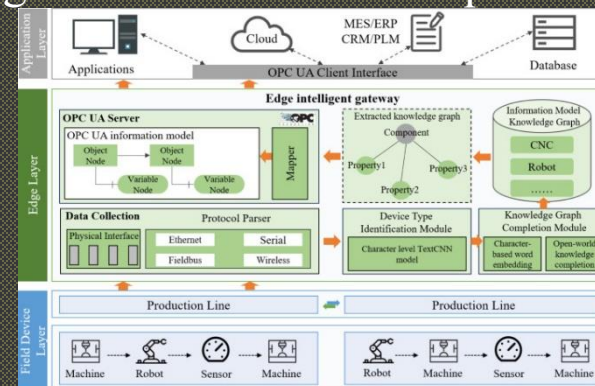
9B.7 OPC UA – Information Model

1. Information Model Overview

- The Information Model defines how data is represented as objects, variables, methods, and their relationships in the Address Space. It allows clients (e.g., HMIs, apps) to understand and interact with servers (e.g., PLCs, devices) in a consistent way, no matter the industry or device type. It is like a map that organizes all the "things" (e.g., motors, sensors) a server manages.
- Example: In a factory, the model might include an object called "ConveyorBelt" with variables like "Speed" (e.g., 50 m/min) and a method "Start."

2. Nodes and NodeClasses

- The Address Space is made up of nodes, each belonging to a NodeClass that defines its type and role. Common NodeClasses include Object, Variable, Method, ObjectType, and ReferenceType.
- Example: A "TemperatureSensor" node (Object) might have a "CurrentTemp" node (Variable) with a value of 25°C, and a "ReadTemp" node (Method) to trigger a reading.



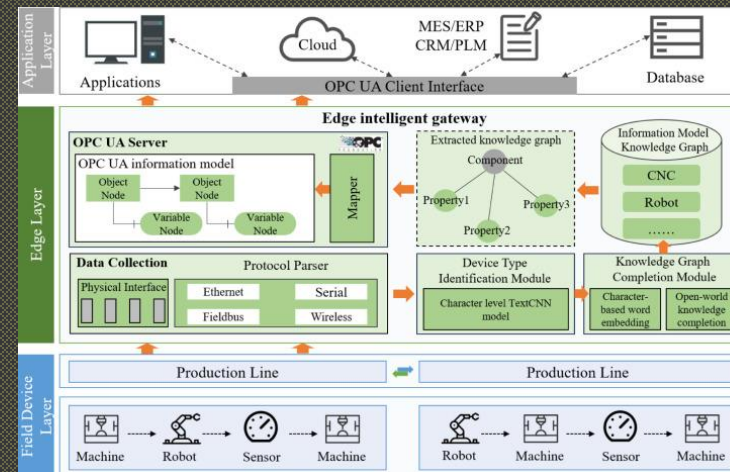
9B.7 OPC UA – Information Model

3. References

- References are links between nodes that show relationships, such as "HasComponent" (part of) or "HasProperty" (attribute of). They create a hierarchical or networked structure.
- Example: A "Motor" object might have a "Speed" variable linked by a "HasProperty" reference, showing Speed is a property of Motor.

4. ObjectTypes and VariableTypes

- These are templates or blueprints that define reusable structures. ObjectTypes describe objects (e.g., a generic "Pump"), while VariableTypes define data types (e.g., a "Temperature" type).
- Example: A "PumpType" ObjectType might include variables like "FlowRate" and "Status," which can be reused for multiple pumps in a plant.



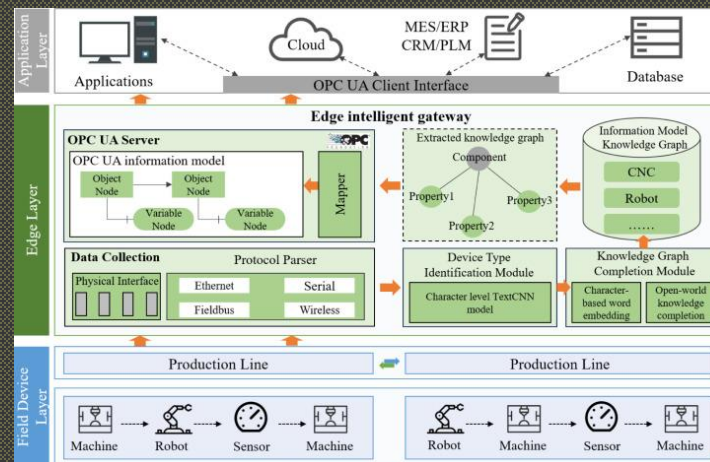
9B.7 OPC UA – Information Model

5. Instances

- Instances are specific objects or variables created from Types. They hold real data and can be unique to a device or process.
- Example: From the "PumpType," you might create "Pump1" (instance) with a "FlowRate" of 100 L/min and "Pump2" with 150 L/min.

6. Predefined Information Models

- OPC UA includes standard models for common industries, like the Data Access Model (for variables) or Alarms and Conditions Model (for alerts).
- Example: The Alarms Model might include a "HighTemperatureAlarm" node that triggers when a sensor exceeds 30°C, notifying operators.



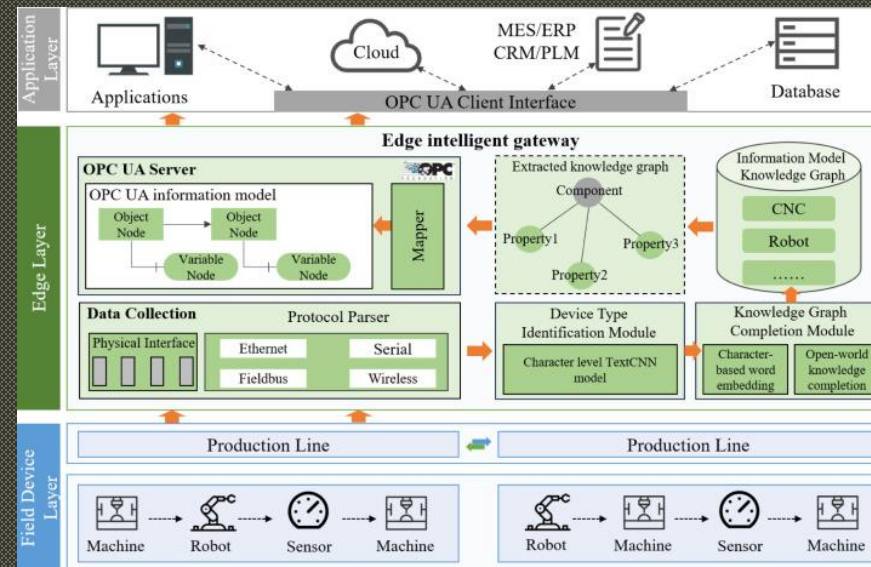
9B.7 OPC UA – Information Model

7. Custom Information Models

- Explanation: Users can create custom models tailored to specific industries or applications, extending the standard ones.
- Example: A food processing plant might add a "BakingOvenModel" with nodes for "DoughWeight" and "BakeTime," unique to its process.

8. Event and Historical Data Modeling

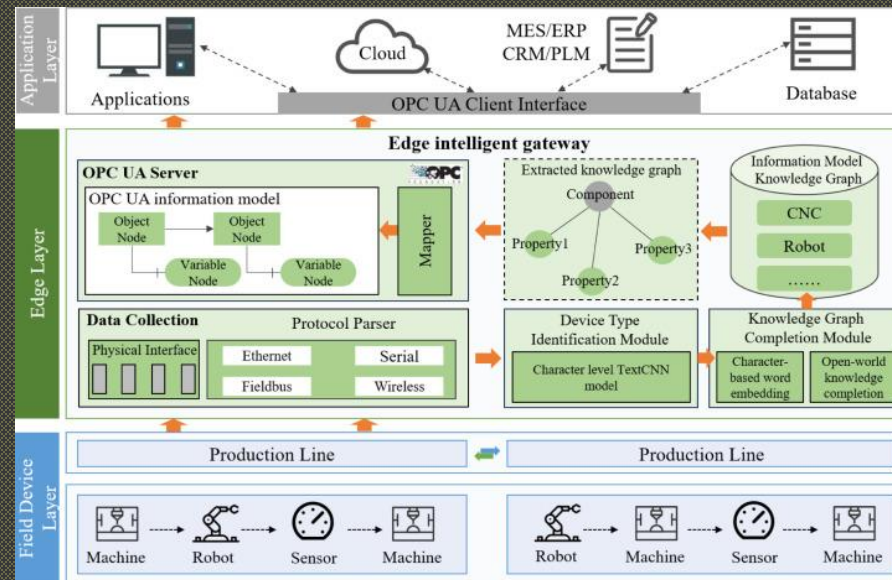
- The model supports events (e.g., alarms) and historical data (e.g., logs), allowing tracking over time.
- Example: A "PowerFailureEvent" node records when a machine stops, and a "HistoricalTemp" node logs temperature every hour.



9B.7 OPC UA – Information Model

How OPC UA Works in Practice

- Process:
 - Define a model (e.g., "MotorModel" with "Speed" and "Start" nodes).
 - Create instances (e.g., "Motor1" with Speed = 100 rpm).
 - Use references to link nodes (e.g., "HasComponent" for Speed under Motor1).
 - Clients browse or read/write data via services (e.g., Read Speed).
- Example Scenario: In a car factory, the Address Space models an assembly line with "RobotArm1" (Object), "Position" (Variable at 50 cm), and "MoveArm" (Method). A client uses the model to monitor and control the arm.



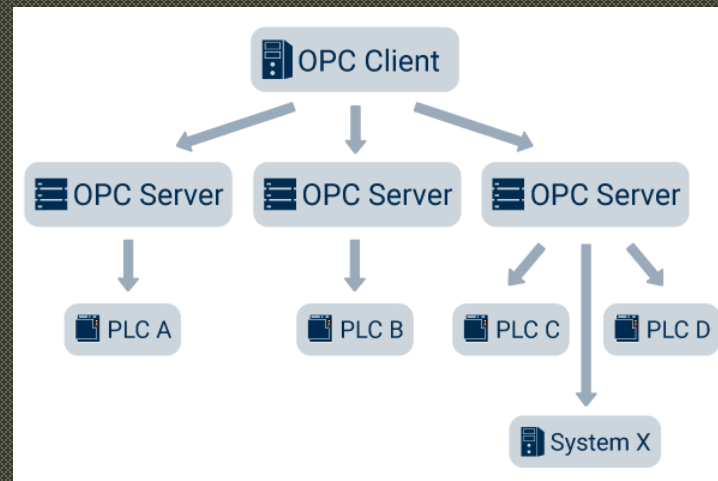
9B.8 OPC UA – Server

OPC Server Overview

- The OPC Server is the core of OPC communication.
- It is software that follows the OPC standard and provides a standardized interface for data exchange.
- Inside the server, it uses the manufacturer's proprietary communication protocol to connect to the device or controller.
- Different companies can provide OPC Servers.

OPC Servers from Hardware Vendors

- Hardware manufacturers often supply their own OPC Server for their devices.
- This allows standardized access to the hardware through OPC.
- These OPC Servers can be:
 - Standalone software installed on a PC, or
 - Embedded software built directly into the device or controller.



Independent (Third-Party) OPC Servers

- Some companies develop independent OPC Servers that support multiple communication protocols.
- They enable OPC communication with devices even if the manufacturer doesn't provide an OPC Server.
- These servers often offer:
 - Wider compatibility (support for many devices and systems)
 - Additional functions and simpler operation
 - Improved stability
- Examples of independent OPC Server providers:
 - Kepware (supports 150+ drivers)
 - Matrikon

9B.8 OPC UA – Client

OPC Client Overview

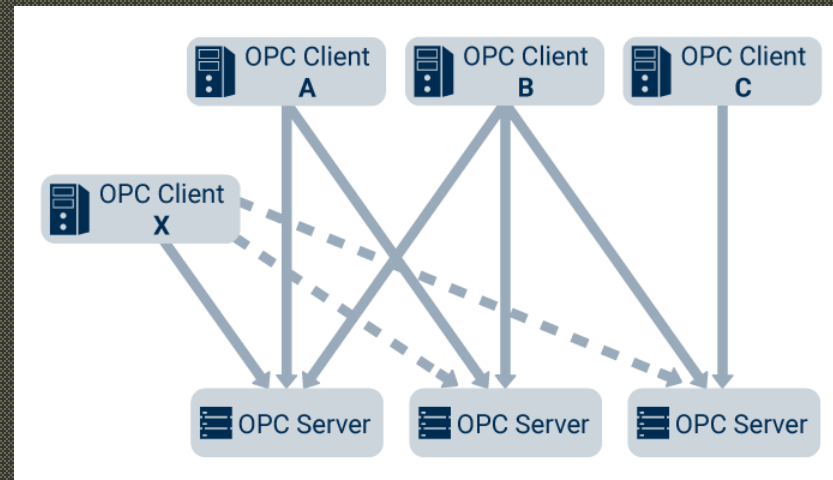
- The OPC Client is the counterpart to the OPC Server.
- It connects to the OPC Server to read and use data provided by the server.
- Because all OPC Servers follow the OPC standard interfaces, any OPC Client can connect to any OPC Server and exchange data in a standardized way.

Applications Acting as OPC Clients

- OPC Clients are usually software applications that need data from industrial systems.
- What they do with the data depends on the specific application.
- Common OPC Client applications include:
 - Visualization and SCADA systems: e.g., Ignition, WinCC, AVEVA, iFIX
 - MES (Manufacturing Execution Systems)

OPC Test Clients

- Free OPC test clients are available to check and troubleshoot OPC Server connections.
- They allow users to:
 - Search for available data points
 - Connect to the server
 - View live data values quickly and easily
- Test clients are useful for verifying OPC data sources independently of other systems.
- A commonly used tool is the Test Client from Unified Automation.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

9B.8 OPC UA – Server/Client

Ignition as Both Server and Client

- Ignition can act as both an OPC UA server and a client at the same time.

1. Server Role (Gateway)

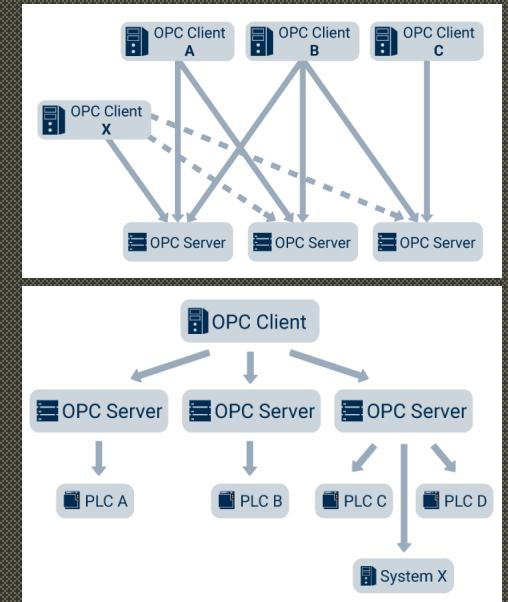
- The Ignition Gateway is the server part.
- It connects to devices and databases to collect and share data.
- Example:
 - The Gateway connects to a PLC and makes the PLC's data available to other systems.
 - Other clients (like HMIs or SCADA systems) can read or write data through this server.

2. Client Role (User Interfaces)

- Ignition's clients are the user interfaces that people use to monitor or control processes.
- These can run on PCs, HMIs, or mobile devices, and they get their data from the Gateway.
- Example:
 - An operator uses an HMI screen built in Ignition Vision or Perspective to view sensor data or start a motor through the Gateway.

3. Combined or Separate Setup

- Both roles can run on one computer (for small systems).
- In large or distributed systems, they can be on different machines:
 - One central server (Gateway) connects to all PLCs.
 - Multiple clients on other PCs or tablets access that data.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

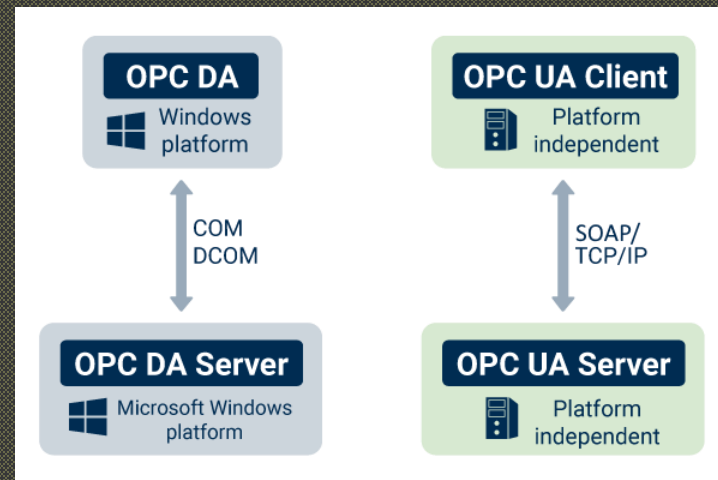
9B.8 OPC Classic vs OPC UA

OPC Classic

- The original OPC standard, now referred to as OPC Classic.
- Main goal: Enable manufacturer-independent data exchange in automation.
- Based on: Microsoft technologies COM and DCOM.
- Limitations:
 - Only worked on Windows systems and networks.
 - Not platform-independent, which restricted use on Linux, web, cloud, IoT, and CPS systems.
- Despite limitations, it was widely used and successful for many years.

OPC UA (Unified Architecture)

- OPC UA is the current and modern OPC standard, the successor to OPC Classic.
- Developed by the OPC Foundation to overcome OPC Classic's limitations.
- Key features:
 - Platform-independent - works on Windows, Linux, and other systems.
 - Interoperable, allows communication across different devices and platforms.
 - Based on web technologies such as TCP/IP and HTTP/SOAP.
- Improves on OPC Classic by combining its proven data exchange concepts with new, extended features for modern automation, cloud, and IoT environments.



SOAP is an XML-based protocol that lets web services exchange structured data.

9B.8 OPC UA PUB/SUB

1. Overview

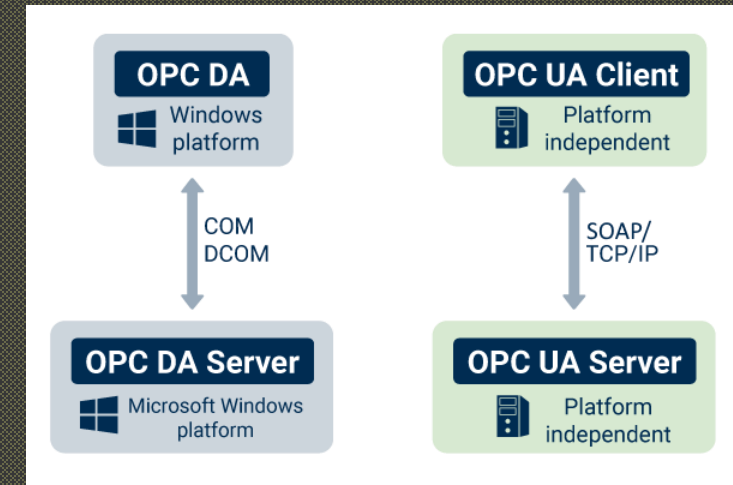
- In OPC UA Pub/Sub, communication participants are divided into:
 - Publisher → Sends (publishes) data
 - Subscriber → Receives (subscribes to) data
- Data exchange happens through a Broker (a message-handling service).
- This approach removes the need for direct 1-to-1 client/server connections, with benefits:
 - Faster communication within the system
 - Reduced processor load on devices

2. OPC Standard Development

- Since its first release in 1994, the OPC standard has continuously evolved to improve access to automation systems.
- The OPC Router can read and write data from OPC data sources and transfer it to other systems.

3. OPC UA Pub/Sub in OPC Router

- The OPC UA Pub/Sub Plug-in adds a new communication method to the OPC Router.
- Based on the Publish-and-Subscribe principle:
 - The OPC Router can act as both:
 - Publisher (Sender) → Sends data
 - Subscriber (Receiver) → Receives data
- This setup allows flexible and efficient data exchange.
- The decoupling of sender and receiver simplifies communication and enables n-to-m (many-to-many) connections instead of one-to-one.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

An OPC Router is a software tool, no-code platform, that links different systems in a factory (like PLCs, databases, and cloud apps) to exchange data.
Example: Software Toolbox.Inc

9B.8 OPC UA Specifications

OPC UA (Open Platform Communications Unified Architecture) defines a comprehensive framework that enables secure, standardized, and platform-independent communication between industrial devices and systems from different vendors. The following specifications describe the main components of this framework:

1. Concepts

- Defines the core ideas and building blocks of OPC UA.
- Includes definitions of data types, objects, variables, and services.
- Establishes the fundamental structure for communication and data exchange.

Example:

A PLC temperature sensor sends a value (as a *Variable*), which belongs to a *Device Object*, and the OPC UA *Service* lets a client read that temperature.

2. Security Model

- Describes how OPC UA protects data and connections.
- Includes:
 - Authentication - verifying user or device identity
 - Encryption - securing transmitted data
 - Access control - restricting actions based on user roles
- Ensures confidentiality, integrity, and availability of communication.

Example:

When an HMI connects to an OPC UA Server, it must log in (authentication), the data is encrypted, and only authorized users can write to the PLC.

9B.8 OPC UA Specifications

3. Address Space Model

- Defines how data is organized and represented inside an OPC UA server.
- Represents all information as nodes (objects, variables, methods, etc.).
- Enables clients to browse, read, or write data in a consistent way.

Example:

The server shows a structured tree: Plant → Line 1 → Motor A → Speed (Variable)

Clients can browse this hierarchy to find data.

4. Services

- Specifies the operations that clients can request from servers.
- Common services include:
 - Read – get data values
 - Write – send or change data
 - Subscribe – monitor data changes
 - Method calls – trigger specific server-side actions

Example:

Read: Get the temperature value from a sensor.

Write: Change a setpoint on a controller.

5. Information Model

- Defines what data means and how it is structured.
- Describes data types, relationships, and hierarchies.
- Allows semantic understanding between different systems, not just raw data exchange.

Example:

A “Pump” Object might have properties like “FlowRate” and “Status”.

This model tells other systems what those values represent, not just the numbers.

9B.8 OPC UA Specifications

6. Mappings

- Specifies how OPC UA messages are transmitted across different networks.
- Maps data and services to transport protocols (e.g., TCP/IP, HTTPS) and encodings (e.g., binary, XML).
- Ensures flexibility and compatibility across platforms.

Example:

On a factory LAN: OPC UA may use Binary over TCP/IP (fast).

Over the internet: OPC UA may use HTTPS with XML (secure, firewall-friendly).

7. Profiles

- Define specific feature sets or functionality groups within OPC UA.
- Allow systems to declare conformance to certain capabilities (e.g., security, data access).
- Help ensure interoperability between devices with similar roles.

Example:

A small sensor might only support the Data Access Profile.

A SCADA system might support Data Access + Alarms + Historical Access.

8. Data Access

- Describes how clients read, write, and subscribe to real-time data values.
- Provides consistent access to live process data (e.g., temperature, pressure, motor speed).
- Supports both cyclic updates and event-driven updates.

Example:

A client continuously reads the temperature, pressure, and valve position from a PLC and updates the HMI screen.

9. Alarms and Conditions

- Defines how OPC UA detects, reports, and manages alarms.
- Includes notification, acknowledgment, and clearing of alarm states.
- Helps operators monitor system health and respond to issues effectively.

Example:

If a motor overheats, the OPC UA server sends an alarm to the operator. The operator acknowledges it once the issue is seen or fixed.

9B.8 OPC UA Specifications

10. Programs

- Specifies how servers can execute and control user-defined programs or sequences.
- Enables automation systems to trigger, monitor, or stop program execution through OPC UA.

11. Historical Access

- Defines how clients can access stored (historical) data from servers.
- Allows retrieval of past values, trends, and events.
- Useful for data analysis, reporting, and optimization.

Example:

An OPC UA client can start, pause, or stop a “Batch Mixing Program” running on a PLC.

Example:

The system retrieves the last 24 hours of temperature data to plot a trend graph or analyze process performance.

12. Discovery

- Describes how clients find and connect to available OPC UA servers.
- Uses discovery servers or network scanning to locate endpoints.
- Simplifies system integration and configuration.

Example:

An HMI uses Local Discovery Server (LDS) to automatically find all OPC UA servers in the plant network and connect to the right one.

13. Aggregates

- Defines how OPC UA can combine data from multiple sources.
- Provides summarized or calculated values (e.g., averages, min/max, counts).
- Useful for data analytics and performance monitoring.

Example:

The OPC UA server calculates the average motor speed every minute from many real-time readings.

9B.8 OPC UA Companion Specifications

1. What They Are

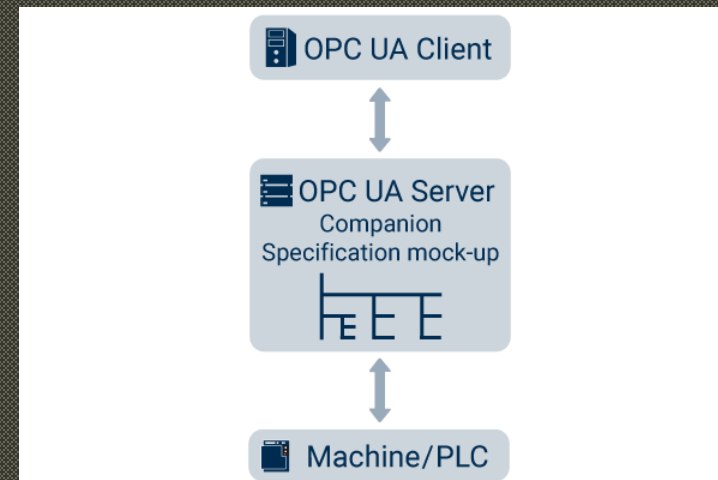
- Companion Specifications are industry-specific information models built on top of the OPC UA Standard Model.
- They define standardized data structures (data points, objects, and parameters) for specific types of machines or systems.

2. Purpose

- Extend the general OPC UA standard to cover specific industry needs.
- Ensure that machines and software from different manufacturers can exchange data in the same format.
- Simplify integration and interoperability between devices in Industry 4.0 environments.

3. Examples

- Injection moulding machines → Euromap 77
- Machine tools / CNC machines → umati
- Robots → Robotics Companion Specification
- RFID and AutoID systems → AutoID
- Many others are available or currently being developed.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

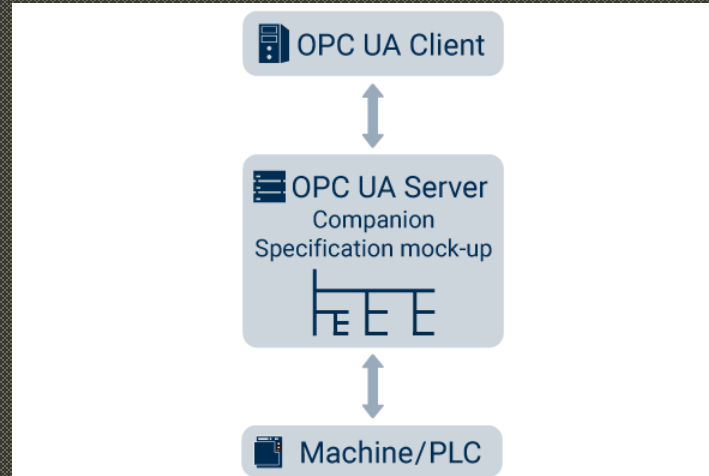
9B.8 OPC UA Companion Specifications

4. Development and Adoption

- Companion Specifications are created by industry groups and working groups in cooperation with the OPC Foundation.
- The OPC Foundation website lists all officially adopted specifications.
- New specifications are continuously being developed and approved.

5. Benefits

- Standardizes both communication and data structure.
 - OPC UA → standardizes how data is exchanged.
 - Companion Specifications → standardize what data is exchanged.
- Makes networking and system integration easier.
- Supports the Industry 4.0 vision by enabling uniform communication between machines from different vendors.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

9B.8 Security in OPC UA

1. Built-in Security from the Start

- OPC UA was designed with security as a core feature from the very beginning.
- Unlike OPC Classic, it is “firewall-friendly”, meaning it can easily work with standard network security tools and firewalls.

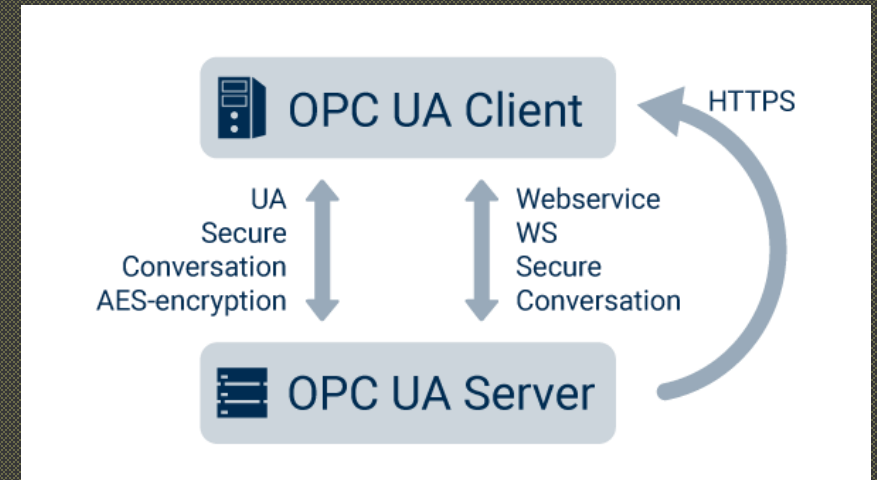
2. Secure and Flexible Communication Protocols

- Supports multiple transport protocols for different needs:
 - Binary protocol over TCP/IP → for fast, real-time communication.
 - SOAP over HTTPS → for cross-platform and web-based communication.

SOAP - Simple Object Access Protocol

3. Strong Data Protection

- Uses encryption (128-bit or 256-bit) to protect data during transmission.
- Adds multiple layers of protection:
 - Message signing – ensures data authenticity.
 - Packet sequencing – prevents message tampering or replay attacks.
 - User authentication – verifies the identity of users and devices.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

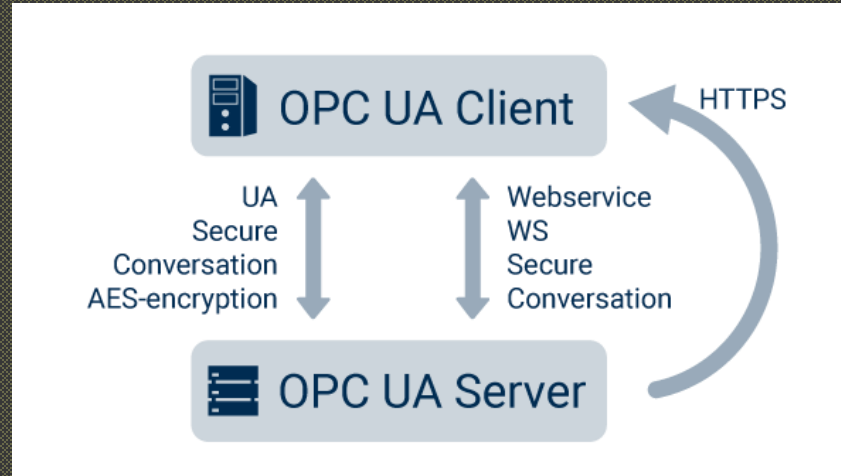
9B.8 Security in OPC UA

4. Certificate-Based Authentication

- Each OPC UA client and server uses digital certificates to verify identity.
- Only clients with approved certificates can connect to the server.
- This helps control access and prevent unauthorized connections.

5. Independent Security Validation

- The German Federal Office for Information Security (BSI) reviewed OPC UA.
- Result: No systemic security vulnerabilities were found.



9B.8 OPC UA Tunneling

1. What is Tunneling?

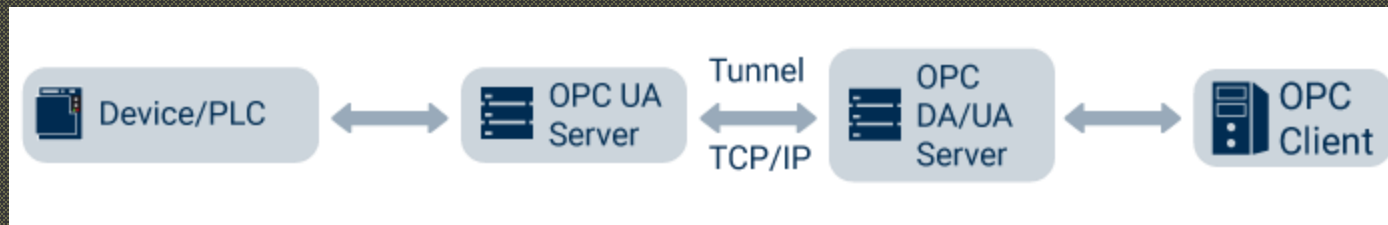
- Tunneling means transferring OPC data between different network segments.
- The term comes from the OPC Classic era, where cross-network communication was difficult due to DCOM technology limitations.

2. Why Tunneling Was Needed (in OPC Classic)

- OPC Classic used Microsoft DCOM, which was not firewall-friendly and hard to configure across networks.
- Communication between devices on different networks often failed or was blocked.

3. How Tunneling Worked

- Software vendors developed tunneling tools to solve this problem.
- These tools:
 - Encapsulated OPC data into simple TCP/IP packets.
 - Allowed data to pass through firewalls safely.
 - Unpacked the data on the target network and presented it again as an OPC Classic Server.



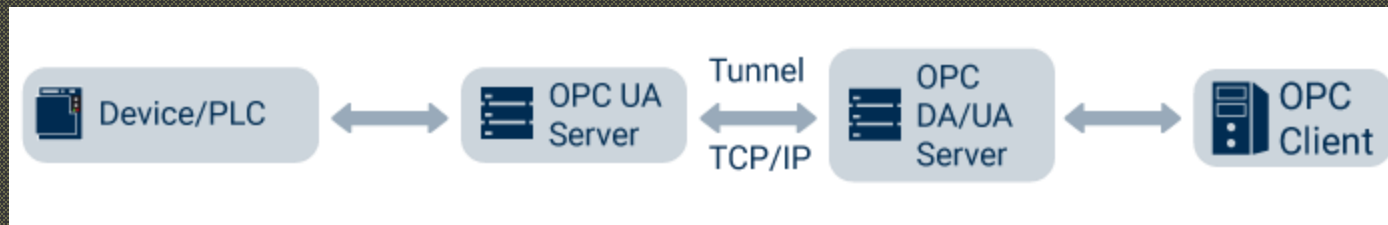
9B.8 OPC UA Tunneling

4. Tunneling in OPC UA

- With OPC UA, tunneling is no longer needed.
- OPC UA uses modern, firewall-friendly protocols (like TCP/IP and HTTPS), enabling secure communication across networks without extra tools.

5. When Tunneling is Still Used

- If a legacy OPC Classic Server still needs to communicate across networks, tunneling can still be applied using an OPC wrapper.
- Example solution:
 - Use an OPC UA Server (e.g., KEPServerEX) to:
 - Act as a client to the existing OPC Classic Server.
 - Expose the data as OPC UA, making it accessible securely across networks.



9B.8 OPC UA over TSN

1. What It Is

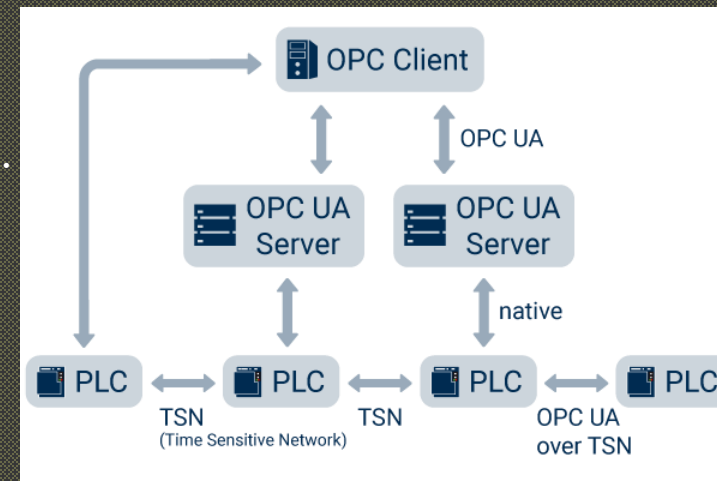
- OPC UA over TSN combines OPC UA (Open Platform Communications Unified Architecture) with TSN (Time-Sensitive Networking).
- It was developed to enable real-time communication at the field level, where controllers, sensors, and actuators need fast, precise data exchange.

2. What TSN Means

- TSN (Time-Sensitive Network) is an extension of standard Ethernet.
- It ensures deterministic communication, meaning data is sent and received at predictable times.
- Ideal for industrial automation, where timing and reliability are critical.

3. Communication Method

- Unlike traditional client-server OPC UA communication, OPC UA over TSN uses the Publisher-Subscriber (Pub/Sub) model.
 - Publisher: Sends real-time data.
 - Subscriber: Receives data exactly when needed.



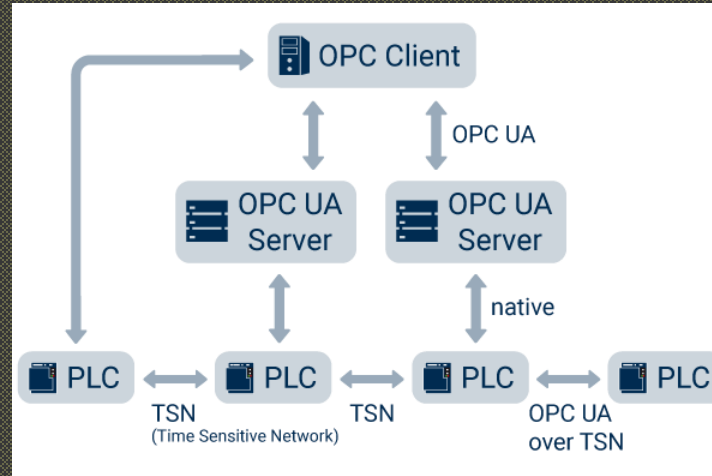
9B.8 OPC UA over TSN

4. Purpose and Benefits

- Designed for real-time control and synchronization between machines and control systems.
- Enables fast, coordinated actions on the factory floor.
- Reduces delays and improves system reliability.

5. Current Use

- Not used in typical non-real-time or standard Ethernet environments.
- Its main role is in industrial field-level communication where real-time performance is essential.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

9B.8 OPC UA PUB/SUB over MQTT

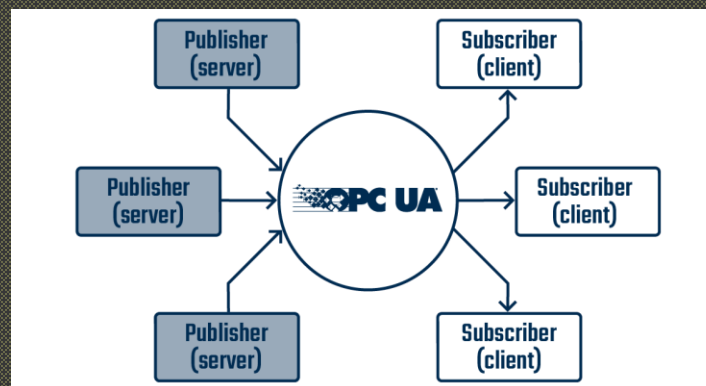
OPC UA Pub/Sub vs MQTT in OPC Router

1. Shared Communication Principle

- Both OPC UA Pub/Sub and MQTT use the Publish/Subscribe (Pub/Sub) communication model.
 - Publisher: Sends (publishes) data.
 - Subscriber: Receives (subscribes to) data.
- The OPC Router can act as both a Publisher and Subscriber with either Plug-in.

2. OPC UA Pub/Sub Plug-in

- Built specifically for OPC UA communication.
- The data structure (objects, variables, types) is already defined and standardized within OPC UA.
- Makes it easy to send or receive OPC UA data between systems and devices.
- Ideal for industrial automation where structured and meaningful data is required.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

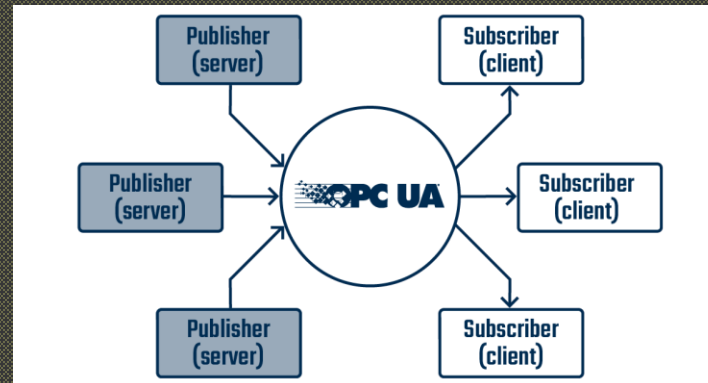
9B.8 OPC UA PUB/SUB over MQTT

3. MQTT Plug-in

- Uses the MQTT protocol, which is common in IoT and cloud applications.
- Message content is usually formatted in XML or JSON.
- To implement OPC UA data in MQTT, extra work is needed to define or translate the data structure manually.
- More flexible but less structured than OPC UA Pub/Sub.

4. Key Difference

- OPC UA Pub/Sub Plug-in:
 - Natively understands OPC UA data models → easy integration.
- MQTT Plug-in:
 - Works with generic message formats (XML/JSON) → requires custom data mapping.



9B.8 OPC XML

1. Purpose

- The OPC XML specification was the OPC Foundation's first step toward platform independence.
- It aimed to move away from Microsoft technologies like COM and DCOM used in OPC Classic.

2. Technology Used

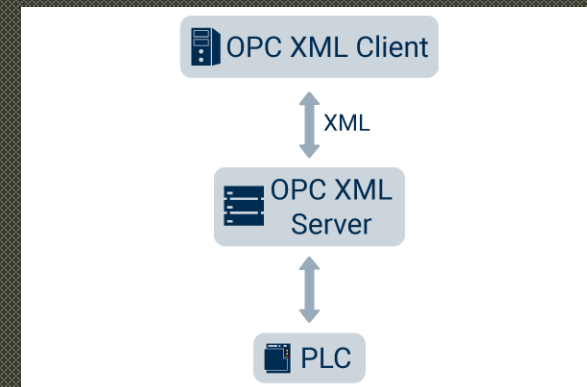
- Based on XML (Extensible Markup Language) and web technologies.
- XML was chosen because it was widely supported and cross-platform at the time.

3. Release

- The OPC XML specification was officially released in 2003.
- It allowed data exchange between systems without depending on Windows.

4. Limitations and Obsolescence

- In 2006, the first version of OPC UA (Unified Architecture) was introduced.
- OPC UA offered:
 - Better performance and security,
 - True platform independence, and
 - More flexible communication models.
- As a result, OPC XML became obsolete and was replaced by OPC UA.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

9B.8 DCOM

1. What COM and DCOM Are

- COM (Component Object Model) is a Microsoft technology that allows different software applications to communicate and share data.
- It is object-oriented and built into the Windows operating system.
- DCOM (Distributed COM) is the network version of COM, enabling communication between computers over a network.

2. Use in OPC Classic

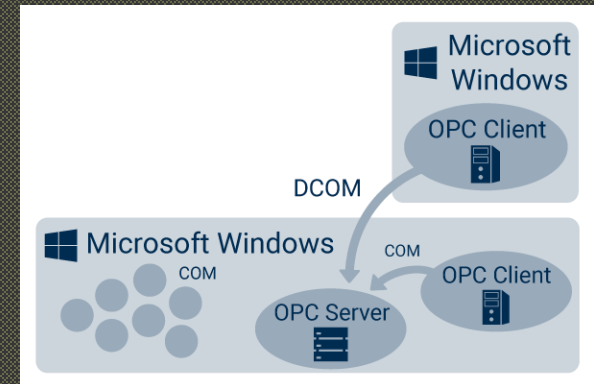
- When OPC Classic was developed, COM/DCOM was the dominant Microsoft technology, so it became the foundation of the OPC standard.
- The OPC Classic Server is a COM component, and OPC Clients connect to it using COM or DCOM.

3. Network Communication via DCOM

- When the OPC Server and Client are on different computers, they must use DCOM for communication.
- DCOM handles:
 - Data exchange over a network
 - Authentication and access control

4. Problems with DCOM

- DCOM is complex and difficult to configure, especially for cross-network connections.
- It uses multiple, dynamic TCP/IP ports, which makes it:
 - Hard to manage through firewalls, and
 - Often blocked by network security policies.
- As a result, DCOM-based OPC connections often failed or were unreliable in real-world applications.



<https://www.opc-router.com/what-is-opc-ua/#OPC-Server>

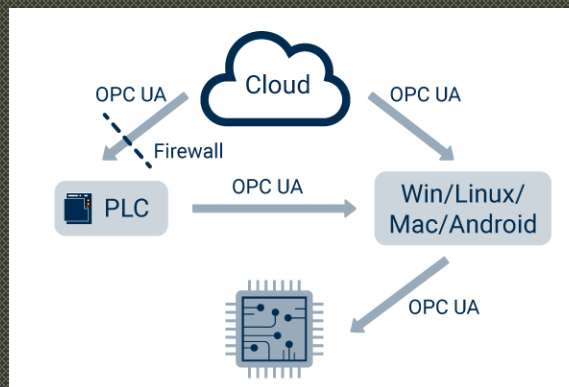
9B.8 Platform independence and interoperability

1. Communication Basis

- OPC UA uses TCP/IP and web-based protocols for communication.
- These are universal network standards, allowing devices to connect easily across different systems.

2. Platform Independence

- OPC UA can run on any operating system, including:
 - Windows
 - Linux
 - UNIX
 - macOS
- Even custom or embedded platforms with a TCP/IP stack can implement an OPC UA Server.



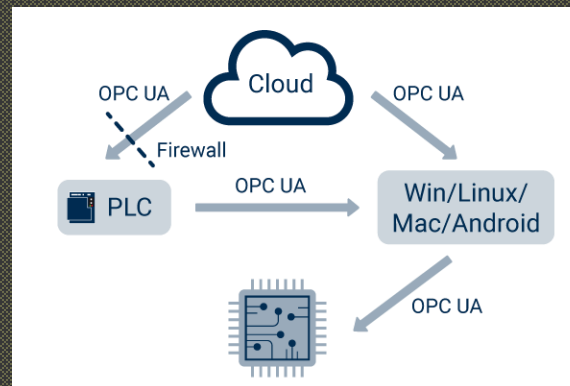
9B.8 Platform independence and interoperability

3. OPC Server Accessibility

- An OPC UA Server makes its data available on the network through these open protocols.
- Clients can connect and exchange data regardless of the operating system used by either side.

4. Benefits

- True platform independence – not limited to Microsoft technologies like OPC Classic was.
- Faster adoption across industries and systems.
- Interoperability – all OPC UA devices and systems can communicate seamlessly, even if they come from different manufacturers or use different platforms.



9B.8 OPC UA and Industry 4.0

1. Background

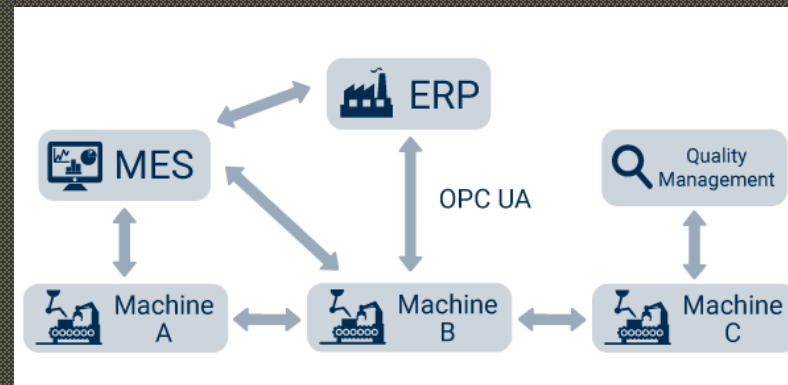
- Industry 4.0 term first used in 2011 by a working group.
- OPC UA was already a well-defined standard before then.

2. Role of OPC UA

- OPC UA is a leading communication protocol for Industry 4.0.
- Provides a common language for intelligent networking of factories and automation systems.

3. Benefits for Industry 4.0

- Enables interoperability between machines and systems from different manufacturers.
- Supports data exchange, monitoring, and control across the entire production network.
- Serves as a key tool for implementing smart factories and industrial automation.



REFERENCES

- Wolfgang Mahnke, Stefan-Helmut Leitner, Matthias Damm, OPC Unified Architecture
- opcfoundation.org

INDUSTRIAL NETWORKING

MENG 4010

Thank you!

Discussions?